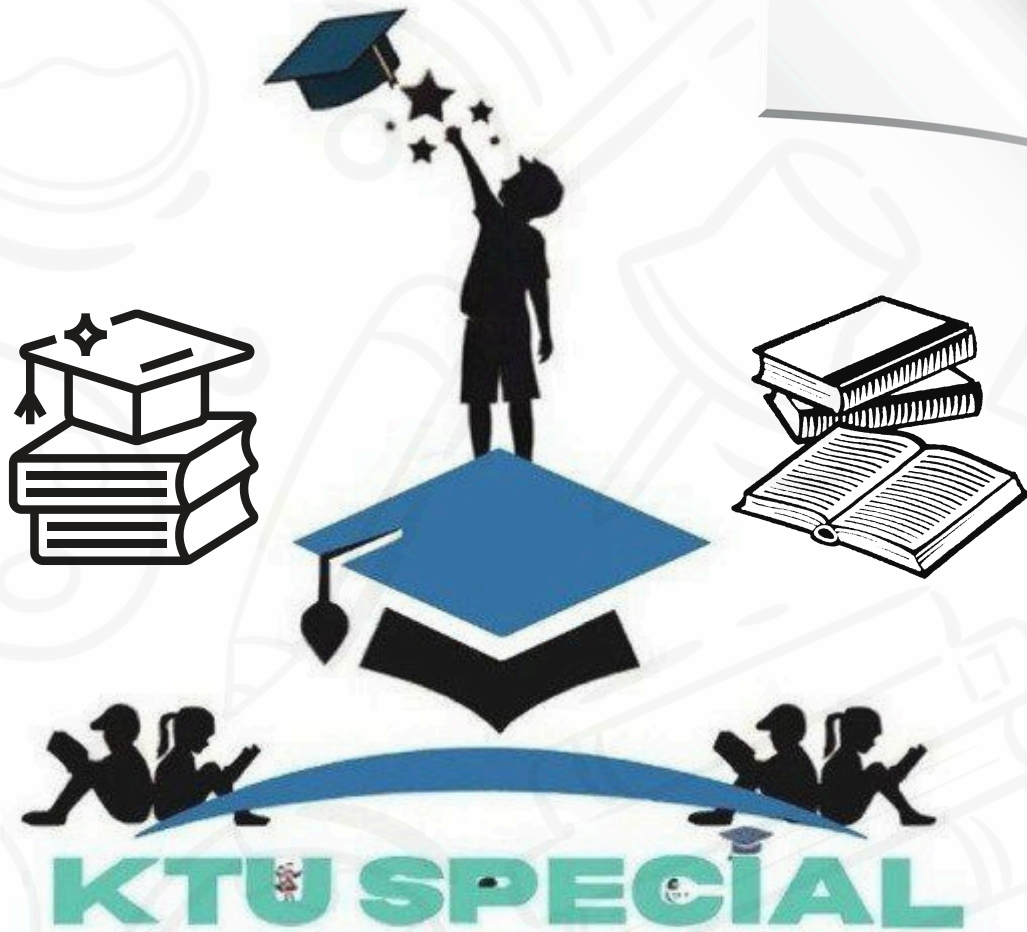




APJ ABDUL KALAM TECHNOLOGICAL UNIVERSITY



KTU SPECIAL

We can together dream the B.tech



**APJ ABDUL KALAM
TECHNOLOGICAL UNIVERSITY**
സാക്ഷാത്കാരം ശാസ്ത്ര സാങ്കേതിക വിദ്യാഭ്യാസം

• **KTU STUDY MATERIALS**

• **SYLLABUS**

• **KTU LIVE NOTIFICATION**

• **SOLVED QUESTION PAPERS**

JOIN WITH US



WWW.KTUSPECIAL.IN



KTUSPECIAL



t.me/ktuspecial1

Syllabus

Pipelining: Introduction to single cycle and multi cycle processors.

Pipelining- Basic and intermediate concepts, Introduction to pipelining, pipeline hazards, structural hazards, data hazards, control hazards, pipelining implementation, RISC V integer pipeline to handle multicycle operations.

Introduction to Single-Cycle and Multi-Cycle Processors

Processors can be designed to execute instructions in different ways, depending on how the instruction execution is divided across clock cycles. Two common approaches are **single-cycle processors** and **multi-cycle processors**.

Single-Cycle Processors

- **Definition:** In a single-cycle processor, each instruction is executed in exactly **one clock cycle**, regardless of its complexity.
- **Key Characteristics:**
 - All operations (fetch, decode, execute, memory access, and write-back) are completed **within** one clock cycle.
 - The clock cycle time is determined by the **slowest instruction**, as every instruction must complete in one cycle. Example: If the longest instruction (e.g., a memory load) takes 100 nanoseconds to complete, the clock cycle for the entire processor must be at least 100 nanoseconds.
 - Temporary registers (like IR, ALUOutput, etc.) are not needed since all operations happen within one cycle.
- **Advantages:**
 - Simple control logic.
 - Easy to design and implement.
- **Disadvantages:**
 - Inefficient for instructions with varying complexity (e.g., memory access vs. ALU operations).
 - Requires a long clock cycle to accommodate the slowest instruction, reducing overall performance.
- **Use Case:** Rarely used in modern processors due to inefficiency but useful for educational purposes.

Multi-Cycle Processors

- **Definition:** In a multi-cycle processor, each instruction is broken into multiple stages, and each is executed in a separate clock cycle.
- **Key Characteristics:**
 - Instructions are divided into stages (e.g., instruction fetch, decode, execute, memory access, write-back).
 - Temporary registers (e.g., IR, ALUOutput, LMD) are used to store intermediate results between cycles.
 - The clock cycle time is shorter, as it only needs to accommodate the **fastest stage** rather than the entire instruction.
 - Different instructions take different numbers of cycles based on their complexity.

- Clock Cycle: Can have a shorter clock cycle because each cycle handles only a portion of an instruction. Example: A memory load instruction might take 5 clock cycles, while an arithmetic instruction might take 3. This allows for better utilization of hardware resources.
- **Advantages:**
 - More efficient use of hardware, as the clock cycle is optimized for individual stages.
 - Reduces the overall CPI (cycles per instruction) for simpler instructions.
- **Disadvantages:**
 - More complex control logic is required to manage the stages.
- **Use Case:** Commonly used in modern processors, especially in pipelined architectures.

Comparison

Aspect	Single-Cycle Processor	Multi-Cycle Processor
Clock Cycle	Long (slowest instruction)	Short (fastest stage)
Instruction Execution	One cycle per instruction	Multiple cycles per instruction
Hardware Utilization	Inefficient	Efficient
Control Logic	Simple	Complex
Performance	Lower	Higher

Concepts of Pipelining

Pipelining is a technique used in computer architecture to improve the throughput of a processor by overlapping the execution of multiple instructions. It is analogous to an assembly line in a factory, where different stages of production are performed simultaneously on different items.

Basic Concepts of Pipelining

1. **Definition:**
 - Pipelining is an implementation technique where multiple instructions are overlapped in execution by dividing the instruction execution process into stages.
2. **Stages of a Basic Pipeline:** A typical instruction execution is divided into the following stages:
 - **Instruction Fetch (IF):** Fetch the instruction from memory.
 - **Instruction Decode/Register Fetch (ID):** Decode the instruction and read the required registers.
 - **Execution/Effective Address Calculation (EX):** Perform the operation or calculate the memory address.
 - **Memory Access (MEM):** Access memory for load/store instructions.
 - **Write-Back (WB):** Write the result back to the register file.

- (Interface registers are used to hold the intermediate output between two stages. These interface registers are also called latch or buffer.)

Instruction number	Clock number								
	1	2	3	4	5	6	7	8	9
Instruction i	IF	ID	EX	MEM	WB				
Instruction $i + 1$		IF	ID	EX	MEM	WB			
Instruction $i + 2$			IF	ID	EX	MEM	WB		
Instruction $i + 3$				IF	ID	EX	MEM	WB	
Instruction $i + 4$					IF	ID	EX	MEM	WB

Figure A.1 Simple RISC pipeline. On each clock cycle, another instruction is fetched and begins its 5-cycle execution. If an instruction is started every clock cycle, the performance will be up to five times that of a processor that is not pipelined. The names for the stages in the pipeline are the same as those used for the cycles in the unpipelined implementation: IF = instruction fetch, ID = instruction decode, EX = execution, MEM = memory access, and WB = write back.

Describe the relation of pipeline stages with execution time (5)

RISC: Five-Stage Pipeline

Consider 5 instructions (n), 5 stages (k)

- The time required to move an instruction one stage down the pipeline is called the **pipeline time**. T_p

$$\begin{aligned}
 \text{Execution Time: } ET_{\text{pipeline}} &= k + n - 1 \text{ cycles or} \\
 &= (k + n - 1) T_p \quad (\text{in ns}) \\
 &= (5+5-1)T_p = 9 T_p \quad (\text{in ns})
 \end{aligned}$$

Ideal CPI of the pipelined processor is '1'

Advantages of Pipelining

1. Instruction throughput increases.
2. Increase in the number of pipeline stages increases the number of instructions executed simultaneously.
3. Faster ALU can be designed when pipelining is used.
4. Pipelined CPU's works at higher clock frequencies than the RAM.
5. Pipelining increases the overall performance of the CPU.

Disadvantages of Pipelining

1. . Designing of the pipelined processor is complex.
2. Instruction latency increases in pipelined processors.
3. The throughput of a pipelined processor is difficult to predict.
4. The longer the pipeline, worse the problem of hazard for branch instructions.

Advantages of RISC Processor , List characteristics of RISC?(3)

1. RISC instructions are **simpler machine instruction**.
2. RISC instructions are **hardwired** to fasten the execution.
3. There are very **fewer** instructions in s RISC instruction set.
4. RISC instruction has simple addressing modes.
5. RISC instruction **executes faster** because most of instruction **operates on processor register** and there is no need to access memory for each instruction.
6. It is **easy to pipeline** RISC instruction as all instruction is of fixed size and opcode and operand are located in the same position in the word.
7. RISC instructions execute **one instruction per clock cycle**.

Disadvantages of RISC Processor

1. RISC instruction size is reduced but more instructions are required to perform an operation when compared with CISC. So, we can say that the length of the program is increased.
2. The machine instructions are **hardwired** in RISC so, it would **cost** if any instruction needs modification.
3. It finds is **difficulty** in processing **complex instruction and complex addressing mode**.
4. RISC instructions do not allow **direct** memory to memory transfer, it requires Load and Store instructions to do so.

Performance of a pipelined processor

Consider a 'k' segment pipeline with clock cycle time as 'Tp'. Let there be 'n' tasks/instructions to be completed in the pipelined processor.

So, speedup (S) of the pipelined processor over non-pipelined processor,

$$S = \text{Performance of pipelined processor} / \text{Performance of Non-pipelined processor}$$

As the performance of a processor is **inversely proportional to the execution time**, we have,

$$\begin{aligned} S &= ET_{\text{non-pipeline}} / ET_{\text{pipeline}} \\ \Rightarrow S &= [n * k * T_p] / [(k + n - 1) * T_p] \\ S &= [n * k] / [k + n - 1] \end{aligned}$$

When the number of tasks 'n' are significantly larger than k, that is, $n \gg k$

$$S = n * k / n$$

$$\text{Speedup } S = k$$

Ideal Speedup = Number of Pipeline Stages (e.g., a 5-stage pipeline can ideally achieve a 5x speedup).

Also, **Efficiency** = Given speed up / Max speed up = S / S_{max}

We know that, $S_{\text{max}} = k$

So, **Efficiency** = S / k

The throughput of a pipeline is determined by how often an instruction exits the pipeline (one per clock cycle in an ideal case).

Throughput = Number of instructions / Total time to complete the instructions

So, **Throughput** = $n / (k + n - 1) * T_p$

Note: The cycles per instruction (CPI) value of an ideal pipelined processor is 1

- Pipelining improves performance by increasing instruction throughput.

Q. "Increasing the number of pipeline stages will decrease the execution time of the program". True or False? Justify your answer.

Answer: False.

Justification:

While increasing the number of pipeline stages can improve performance, it does not always decrease the execution time of a program. Here's why:

☒ Advantages of More Pipeline Stages:

Higher clock speed: Each stage does less work, so the clock cycle time can be reduced.

Increased instruction throughput: More instructions can be processed simultaneously in different stages.

☒ But it also introduces drawbacks:

Increased pipeline overhead:

Each stage needs an inter-stage buffer (register), adding latency.

More stages = more overhead due to these pipeline registers.

Higher chance of hazards:

Data hazards, control hazards, and structural hazards become more difficult to handle.

Branch instructions, for example, can cause more pipeline stalls in a deeper pipeline.

Diminishing returns:

After a certain point, increasing the number of stages doesn't significantly improve performance.

Eventually, the overhead and complexity outweigh the benefits.

Increased penalty on pipeline stalls and flushes:

In deeper pipelines, when a stall or branch misprediction occurs, more stages have to be flushed, wasting more cycles.

Types of pipeline

- Uniform delay/ Linear pipeline

In this type of pipeline, all the stages will take same time (T_p) to complete an operation.

- Non-Uniform/Non-linear delay pipeline

In this type of pipeline, different stages take different time to complete an operation.

In this type of pipeline, **Pipeline Time (T_p) = Maximum(Stage Delay)**

For example, if there are 4 stages with delays, 1 ns, 2 ns, 3 ns, and 4 ns, then

$$T_p = \text{Maximum}(1 \text{ ns}, 2 \text{ ns}, 3 \text{ ns}, 4 \text{ ns}) = 4 \text{ ns}$$

If buffers are included between the stages,

$$T_p = \text{Maximum}(\text{Stage delay}) + \text{Buffer delay}$$

Q : Consider a 4 segment pipeline with stage delays (2 ns, 8 ns, 3 ns, 10 ns). Find the time taken to execute 100 tasks in the above pipeline.

Solution : As the above pipeline is a non-linear pipeline,

$$T_p = \max(2, 8, 3, 10) = 10 \text{ ns}$$

$$\text{We know that } ET_{\text{pipeline}} = (k + n - 1) T_p = (4 + 100 - 1) 10 \text{ ns} = 1030 \text{ ns}$$

CLASSIFICATION OF PIPELINE PROCESSORS

- **Arithmetic Pipelining:** An arithmetic pipeline in computer architecture is a technique that speeds up arithmetic operations by breaking them down into smaller, pipelined stages. Each stage performs a specific part of the calculation, allowing multiple operations to be processed concurrently.
- **Instruction Pipelining:** The execution of stream of instructions can be pipelined by overlapping the execution of current instruction with subsequent instructions.
- **Processor Pipelining:** Pipeline processing of the same datastream by a cascade of processors, each of which processes a specific task. The data stream passes the first processor with the results stored in memory block which is also accessible by the second processor. The second processor then passes the refined results to the third and soon.

Q. Consider an instruction pipeline with four stages with the stage delays 5 nsec, 6 nsec, 11 nsec, and 8 nsec respectively. The delay of an inter-stage register stage of the pipeline is 1 nsec. What is the approximate speedup of the pipeline in the steady state under ideal conditions as compared to the corresponding non-pipelined implementation?

◆ 1. Non-pipelined execution time per instruction

In a non-pipelined processor, the Pipeline time is the **sum** of all stage delays:

$$T_p = 5 + 6 + 11 + 8 = 30 \text{ ns}$$

$$ET_{\text{non-pipeline}} = n * k * T_p$$

ET is directly proportional to T_p

◆ 2. Pipelined clock cycle time

In a pipelined processor, the clock cycle is determined by the **longest stage delay + register delay**.

$$T_p = \max(5, 6, 11, 8) + 1 = 11 + 1 = 12 \text{ ns}$$

$$ET_{\text{pipeline}} = (k + n - 1) T_p = (k + n - 1) * 12$$

◆ 3. Speedup Calculation

Speedup is the ratio of $ET_{\text{non-pipelined}}$ to $ET_{\text{pipelined}}$

ie

$$\text{Speedup} = ET_{\text{non-pipelined}} / ET_{\text{pipelined}} = 30/12 = 2.5$$

Stages of a Basic Pipeline-Timeline diagram



APJ ABDUL KALAM
TECHNOLOGICAL UNIVERSITY

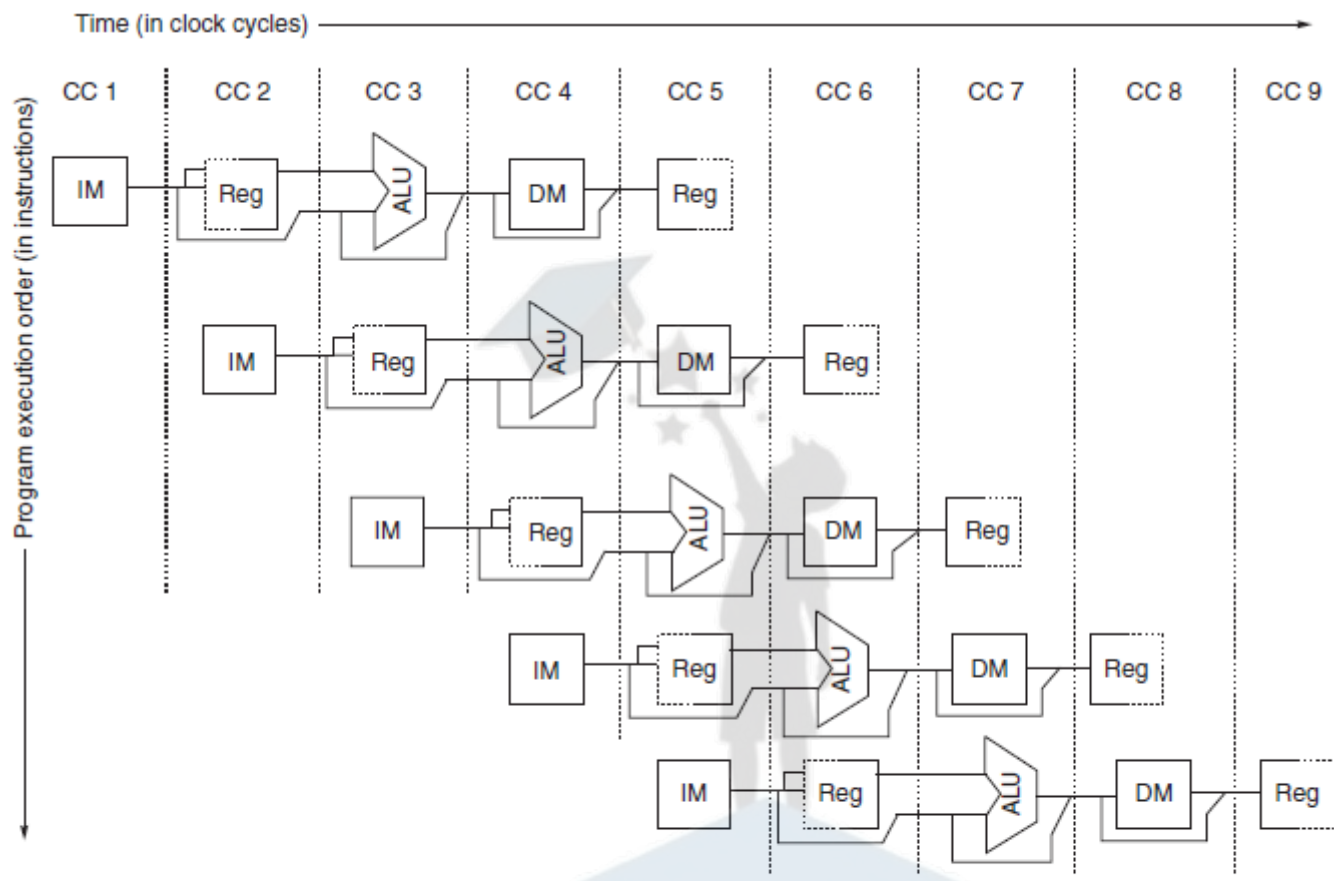


Figure C.2 The pipeline can be thought of as a series of data paths shifted in time. This shows the overlap among the parts of the data path, with clock cycle 5 (CC 5) showing the steady-state situation. Because the register file is used as a source in the ID stage and as a destination in the WB stage, it appears twice. We show that it is read in one part of the stage and written in another by using a solid line, on the right or left, respectively, and a dashed line on the other side. The abbreviation IM is used for instruction memory, DM for data memory, and CC for clock cycle.



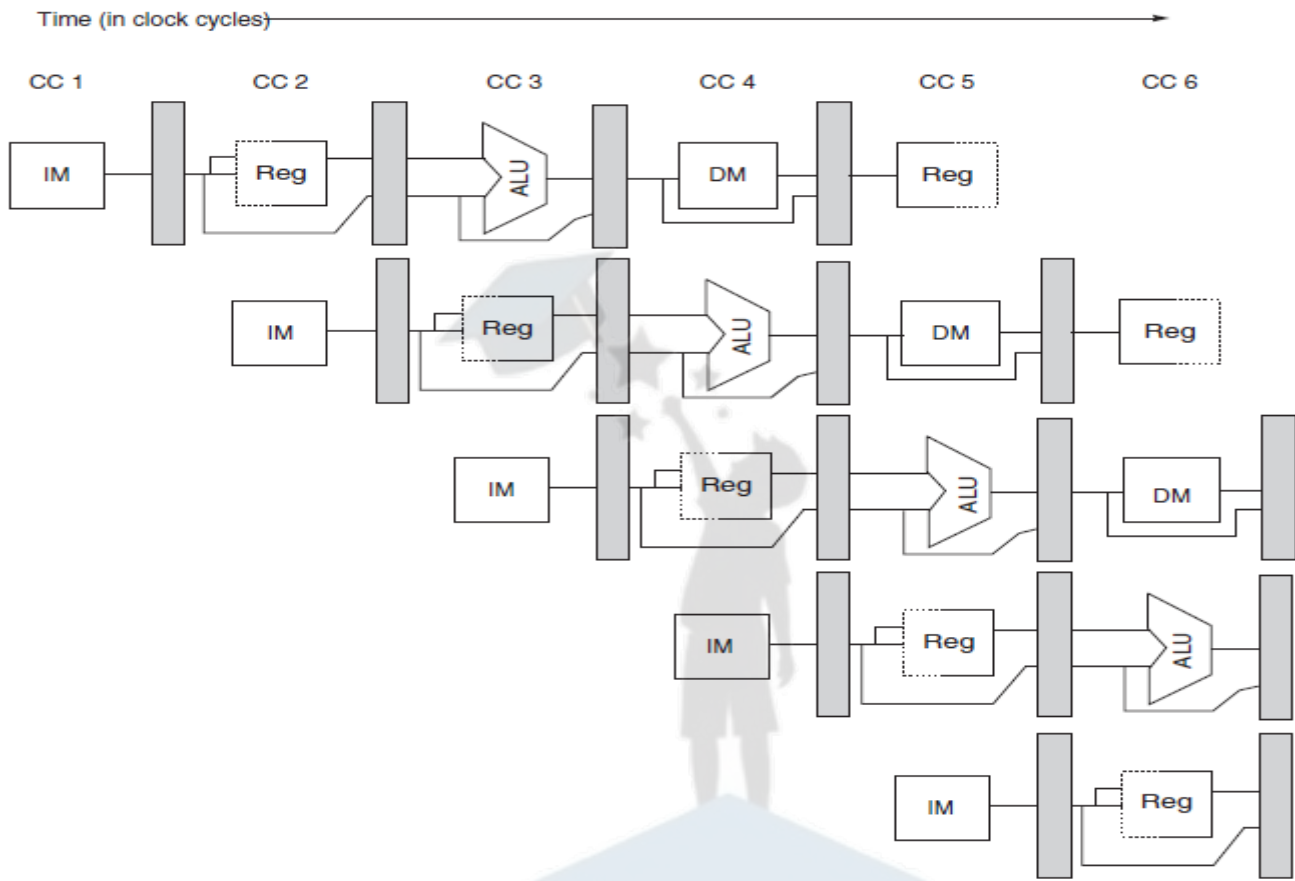


Figure C.3 A pipeline showing the pipeline registers between successive pipeline stages. Notice that the registers prevent interference between two different instructions in adjacent stages in the pipeline. The registers also play the critical role of carrying data for a given instruction from one stage to the other. The edge-triggered property of registers—that is, that the values change instantaneously on a clock edge—is critical. Otherwise, the data from one instruction could interfere with the execution of another!

Basic Performance Issues in Pipelining

Consider the unpipelined processor in the previous section. Assume that it has a 1 ns clock cycle and that it uses 4 cycles for ALU operations and branches and 5 cycles for memory operations. Assume that the relative frequencies of these operations are 40%, 20%, and 40%, respectively. Suppose that due to clock skew and setup, pipelining the processor adds 0.2 ns of overhead to the clock. Ignoring any latency impact, how much speedup in the instruction execution rate will we gain from a pipeline?

The average instruction execution time on the unpipelined processor is

$$\begin{aligned}
 \text{Average instruction execution time} &= \text{Clock cycle} \times \text{Average CPI} \\
 &= 1 \text{ ns} \times [(40\% + 20\%) \times 4 + 40\% \times 5] \\
 &= 1 \text{ ns} \times 4.4 \\
 &= 4.4 \text{ ns}
 \end{aligned}$$

In the pipelined implementation, the clock must run at the speed of the slowest stage plus overhead, which will be 1 + 0.2 or 1.2 ns; this is the average instruction execution time. Thus, the speedup from pipelining is

$$\begin{aligned}
 \text{Speedup from pipelining} &= \frac{\text{Average instruction time unpipelined}}{\text{Average instruction time pipelined}} \\
 &= \frac{4.4 \text{ ns}}{1.2 \text{ ns}} = 3.7 \text{ times}
 \end{aligned}$$

The Major Hurdle of Pipelining—Pipeline Hazards

1. Structural hazards arise from resource conflicts when the hardware cannot support all possible combinations of instructions simultaneously in overlapped execution.
2. Data hazards arise when an instruction depends on the results of a previous instruction in the pipeline.
3. Control hazards arise from the pipelining of branches and other instructions that change the PC.

Hazards in pipelines can make it necessary to stall the pipeline. Avoiding a hazard often requires that some instructions in the pipeline be allowed to proceed while others are delayed.

Performance of pipeline with stalls

Speed Up (S) = Performance_{pipeline} / Performance_{non-pipeline}

=> S = Average Execution Time_{non-pipeline} / Average Execution Time_{pipeline}

=> S = CPI_{non-pipeline} * Cycle Time_{non-pipeline} / CPI_{pipeline} * Cycle Time_{pipeline}

Ideal CPI of the pipelined processor is '1'.

But due to stalls, it becomes greater than '1'.

=>

S = CPI_{non-pipeline} * Cycle Time_{non-pipeline} / (1 + Number of stalls per Instruction) * Cycle Time_{pipeline}

As Cycle Time_{non-pipeline} / Cycle Time_{pipeline} = Pipeline Depth

Speed Up (S) = CPI_{non-pipeline} / (1 + Number of stalls per instruction) * Pipeline Depth

3. Key Features:

- o Each stage operates in parallel on different instructions.
- o Instructions enter the pipeline sequentially, and each stage processes one instruction per clock cycle.

4. Pipeline Throughput:

- o The throughput of a pipeline is determined by how often an instruction exits the pipeline (one per clock cycle in an ideal case).
- o The time required to move an instruction one stage down the pipeline is called the **pipeline cycle time**.

5. Pipeline Performance Metrics:

- o **Throughput:** Number of instructions completed per unit time.
- o **Latency:** Time taken to execute a single instruction from start to finish.
- o **CPI (Cycles Per Instruction):** Average number of cycles required per instruction.
 - Ideal CPI for a pipelined processor is 1, but hazards and stalls increase it.

6. Pipeline Efficiency:

- o **Pipeline Efficiency (%)** = (Number of Instructions Completed / Total Cycles) × 100
- o Efficiency decreases with more stalls and hazards.

7. Pipeline Depth:

- o Increasing the number of pipeline stages (pipeline depth) can improve clock speed but also increases the complexity of handling hazards.
- o Deeper pipelines have higher branch penalties and are more sensitive to hazards.

8. Pipeline Hazards: Situations that prevent the next instruction from executing in its designated cycle.

- **Structural Hazards:** Resource conflicts (e.g., single memory for instructions and data).
- **Data Hazards:** Dependencies between instructions (e.g., RAW, WAR, WAW).
- **Control Hazards:** Branches and jumps that change the program counter.

Intermediate Concepts of Pipelining

1. Pipeline Hazards in Detail:

◦ Structural Hazards:

- Occur when hardware resources are insufficient to support all instructions in the pipeline.
- Example: A single memory unit being accessed simultaneously for instruction fetch and data access.

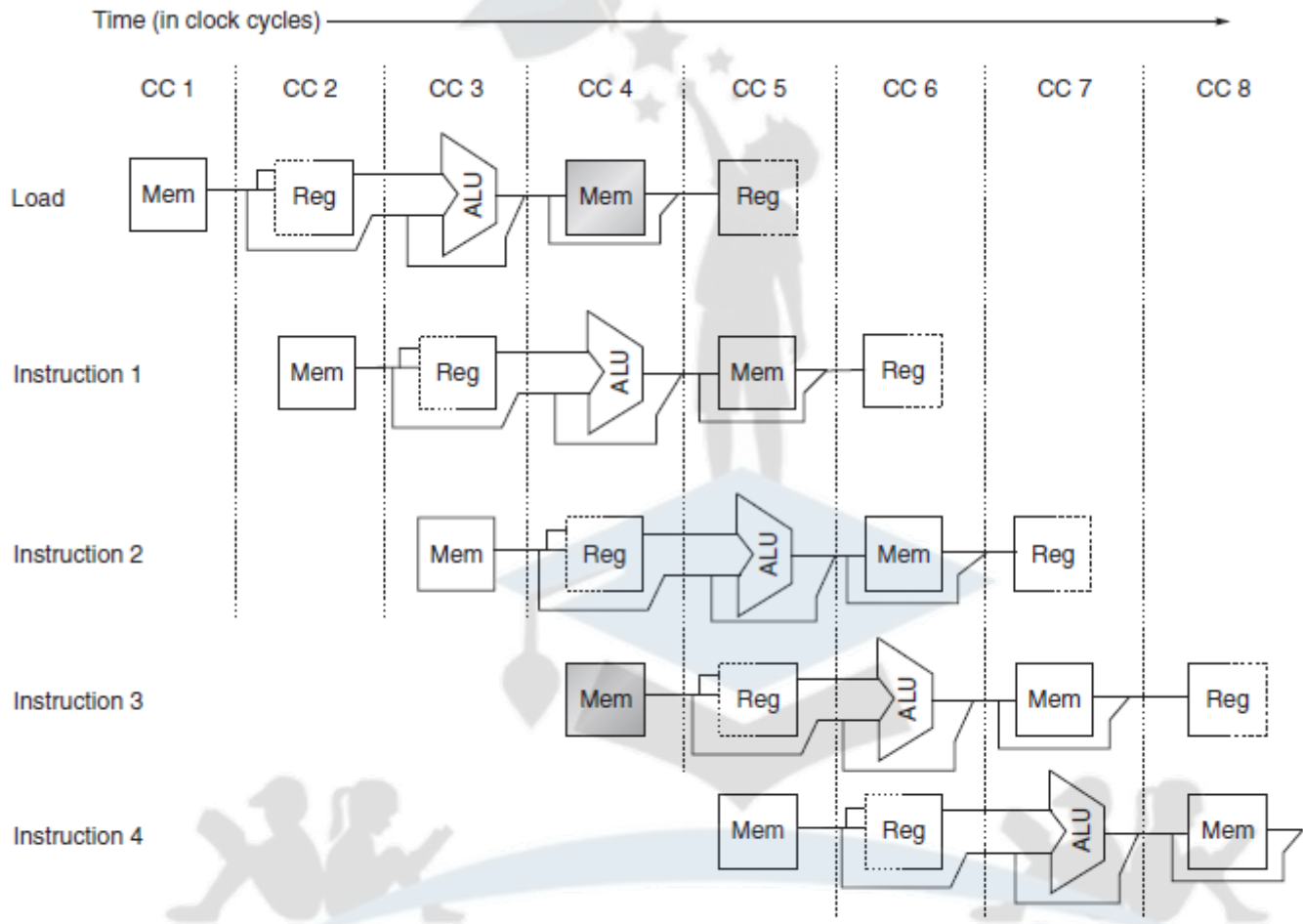


Figure C.4 A processor with only one memory port will generate a conflict whenever a memory reference occurs. In this example the load instruction uses the memory for a data access at the same time instruction 3 wants to fetch an instruction from memory.

Techniques to Handle Structural Hazards:

- Add more hardware resources (e.g., separate instruction and data memory). To resolve this hazard, we stall the pipeline for 1 clock cycle when the data memory access occurs. A stall is commonly called a pipeline bubble or just bubble, since it floats through the pipeline taking space but carrying no useful work. A stall (or bubble) is introduced into the pipeline when an instruction cannot proceed due to a hazard.
- Stalls reduce the pipeline's efficiency and increase CPI.

Let's see how much the load structural hazard might cost. Suppose that data references constitute 40% of the mix, and that the ideal CPI of the pipelined processor, ignoring the structural hazard, is 1. Assume that the processor with the structural hazard has a clock rate that is 1.05 times higher than the clock rate of the processor without the hazard. Disregarding any other performance losses, is the pipeline with or without the structural hazard faster, and by how much?

There are several ways we could solve this problem. Perhaps the simplest is to compute the average instruction time on the two processors:

$$\text{Average instruction time} = \text{CPI} \times \text{Clock cycle time}$$

Instruction	Clock cycle number									
	1	2	3	4	5	6	7	8	9	10
Load instruction	IF	ID	EX	MEM	WB					
Instruction $i + 1$		IF	ID	EX	MEM	WB				
Instruction $i + 2$			IF	ID	EX	MEM	WB			
Instruction $i + 3$				Stall	IF	ID	EX	MEM	WB	
Instruction $i + 4$						IF	ID	EX	MEM	WB
Instruction $i + 5$							IF	ID	EX	MEM
Instruction $i + 6$								IF	ID	EX

Figure C.5 A pipeline stalled for a structural hazard—a load with one memory port. As shown here, the load instruction effectively steals an instruction-fetch cycle, causing the pipeline to stall—no instruction is initiated on clock cycle 4 (which normally would initiate instruction $i + 3$). Because the instruction being fetched is stalled, all other instructions in the pipeline before the stalled instruction can proceed normally. The stall cycle will continue to pass through the pipeline, so that no instruction completes on clock cycle 8. Sometimes these pipeline diagrams are drawn with the stall occupying an entire horizontal row and instruction 3 being moved to the next row; in either case, the effect is the same, since instruction $i + 3$ does not begin execution until cycle 5. We use the form above, since it takes less space in the figure. Note that this figure assumes that instructions $i + 1$ and $i + 2$ are not memory references.

Since it has no stalls, the average instruction time for the ideal processor is simply the Clock cycle time_{ideal}. The average instruction time for the processor with the structural hazard is

$$\begin{aligned} \text{Average instruction time} &= \text{CPI} \times \text{Clock cycle time} \\ &= (1 + 0.4 \times 1) \times \frac{\text{Clock cycle time}_{\text{ideal}}}{1.05} \\ &= 1.3 \times \text{Clock cycle time}_{\text{ideal}} \end{aligned}$$

Clearly, the processor without the structural hazard is faster; we can use the ratio of the average instruction times to conclude that the processor without the hazard is 1.3 times faster.

◦ Data Hazards:

- **RAW (Read After Write):** An instruction depends on the result of a previous instruction.
- **WAR (Write After Read):** A later instruction writes to a register before an earlier instruction reads it.
- **WAW (Write After Write):** Two instructions write to the same register in an incorrect order.

Consider the pipelined execution of these instructions:

DADD R1,R2,R3
 DSUB R4,R1,R5
 AND R6,R1,R7
 OR R8,R1,R9
 XOR R10,R1,R11

All the instructions after the DADD use the result of the DADD instruction. As shown in Figure C.6, the DADD instruction writes the value of R1 in the WB pipe stage, but the DSUB instruction reads the value during its ID stage. This problem is called a data hazard. Unless precautions are taken to prevent it, the DSUB instruction will read the wrong value and try to use it.

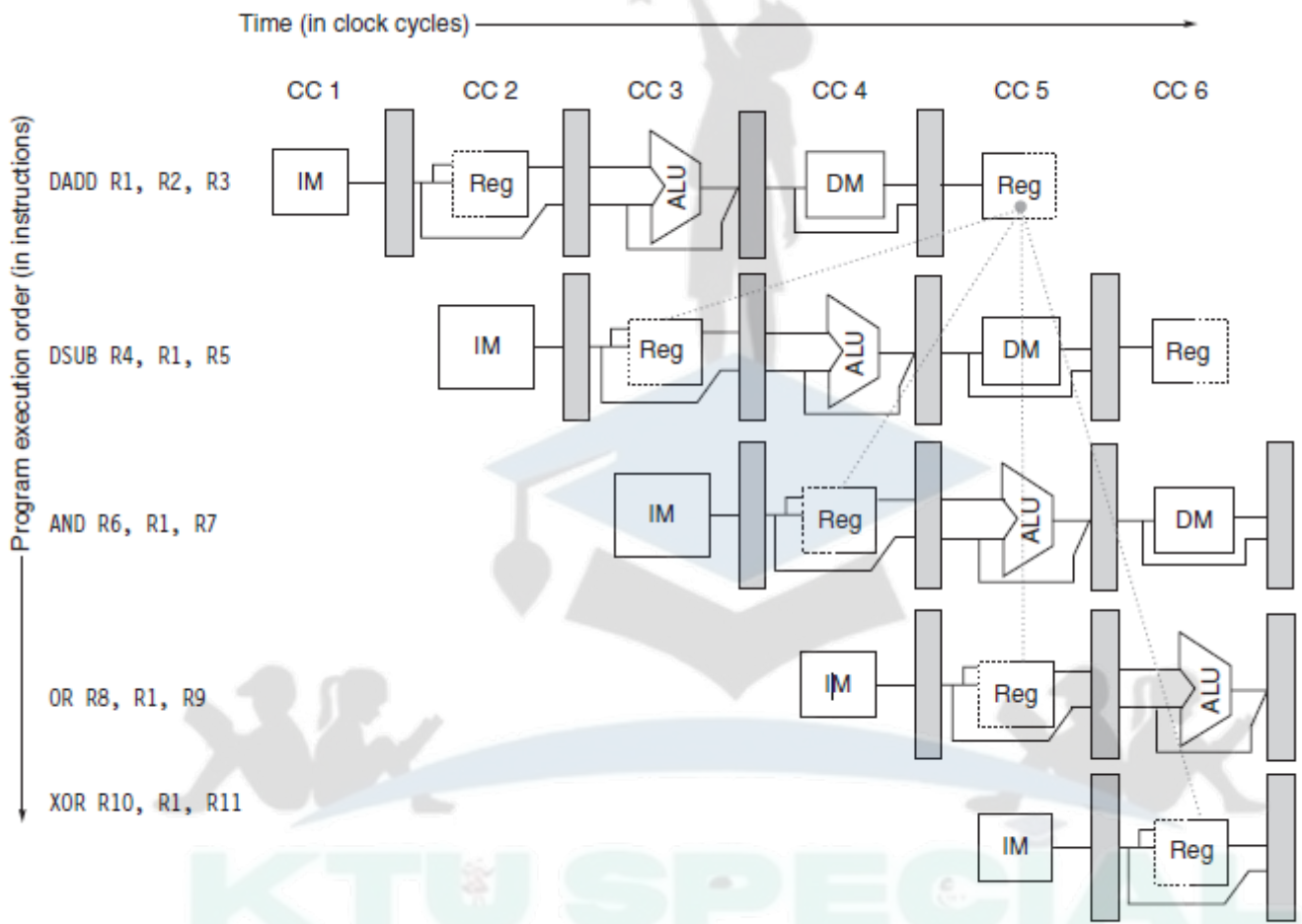


Figure C.6 The use of the result of the DADD instruction in the next three instructions causes a hazard, since the register is not written until after those instructions read it.

○ Techniques to solve Data Hazards:

- **Forwarding (Bypassing):** Pass the result of an instruction directly to a dependent instruction without waiting for it to be written back.
- **Pipeline Interlocks:** Stall the pipeline until the hazard is resolved.

1. Minimizing Data Hazard Stalls by Forwarding

1. The ALU result from both the EX/MEM and MEM/WB pipeline registers is always fed back to the ALU inputs.
2. If the forwarding hardware detects that the previous ALU operation has written the register corresponding to a source for the current ALU operation, control logic selects the forwarded result as the ALU input rather than the value read from the register file.

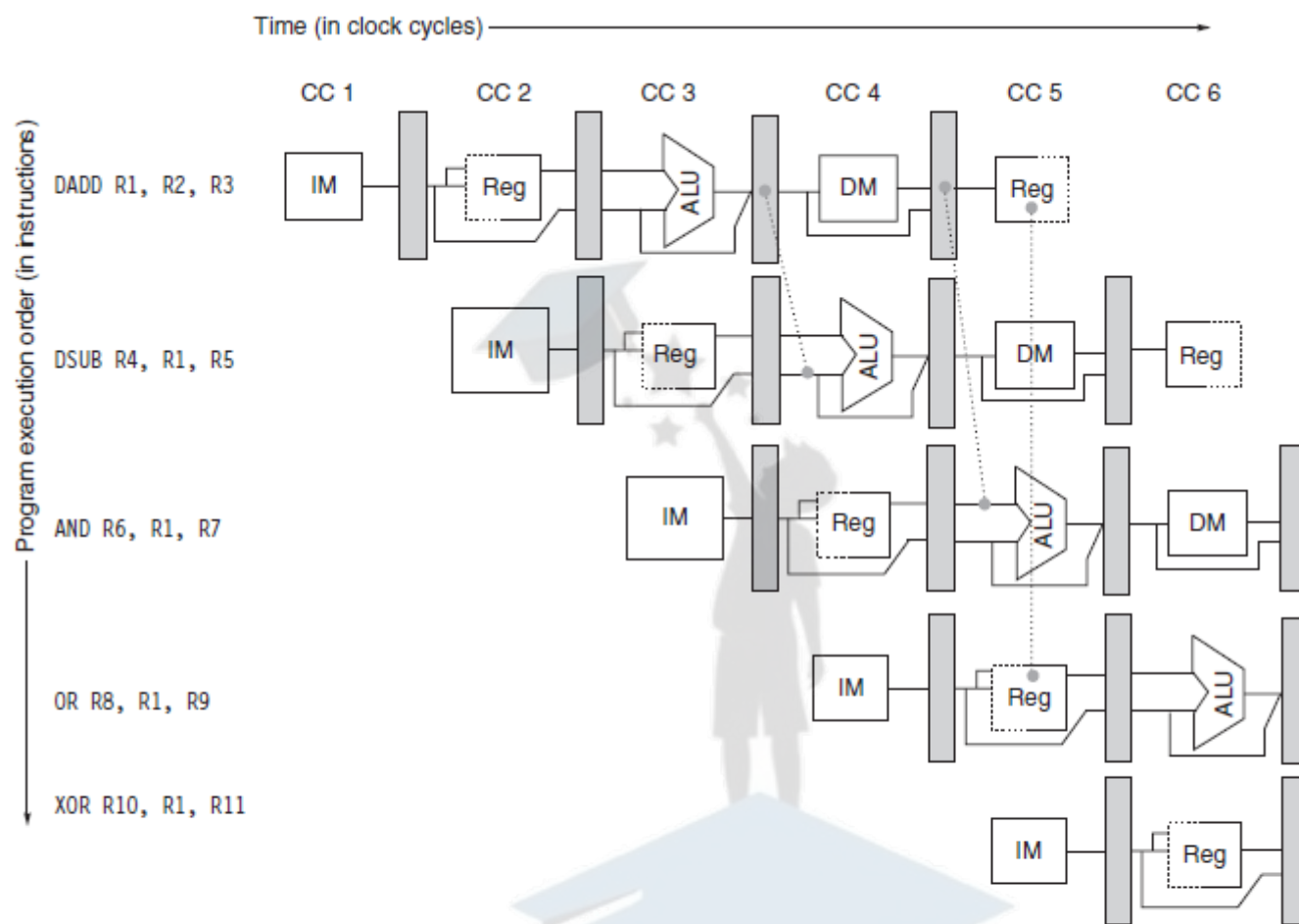
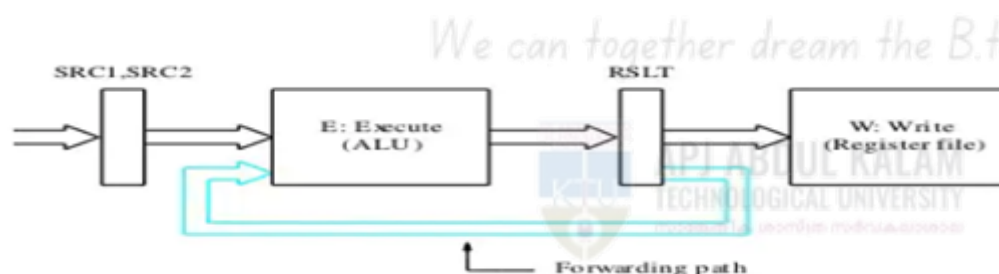


Figure C.7 A set of instructions that depends on the DADD result uses forwarding paths to avoid the data hazard. The inputs for the DSUB and AND instructions forward from the pipeline registers to the first ALU input. The OR receives its result by forwarding through the register file, which is easily accomplished by reading the registers in the second half of the cycle and writing in the first half, as the dashed lines on the registers indicate. Notice that the forwarded result can go to either ALU input; in fact, both ALU inputs could use forwarded inputs from either the same pipeline register or from different pipeline registers. This would occur, for example, if the AND instruction was AND R6, R1, R4.

What is operand forwarding? What is its significance?

Operand Forwarding :

It adds special circuitry to the pipeline. This method works because it takes **less time** for the required values to travel through a wire straight from ALU than it does for a pipeline segment to compute its result.



Take, for example, the following sequence:

DADD R1,R2,R3

LD R4,0(R1)

SD R4,12(R1)

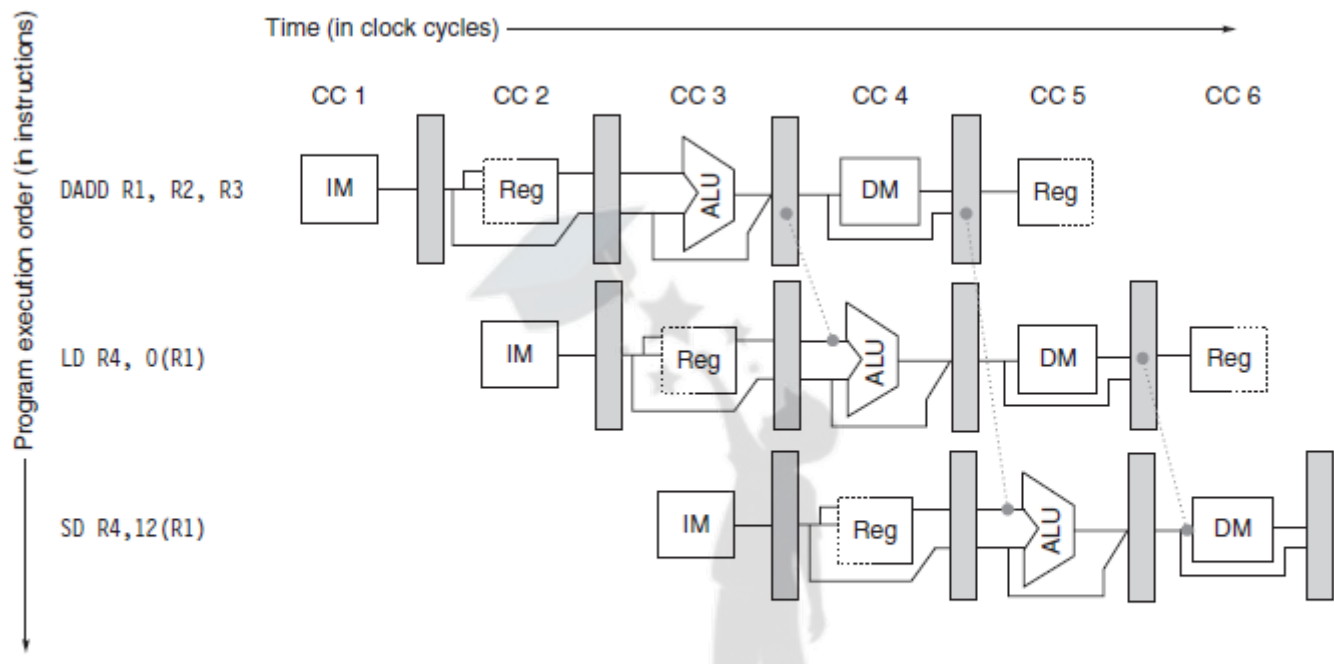


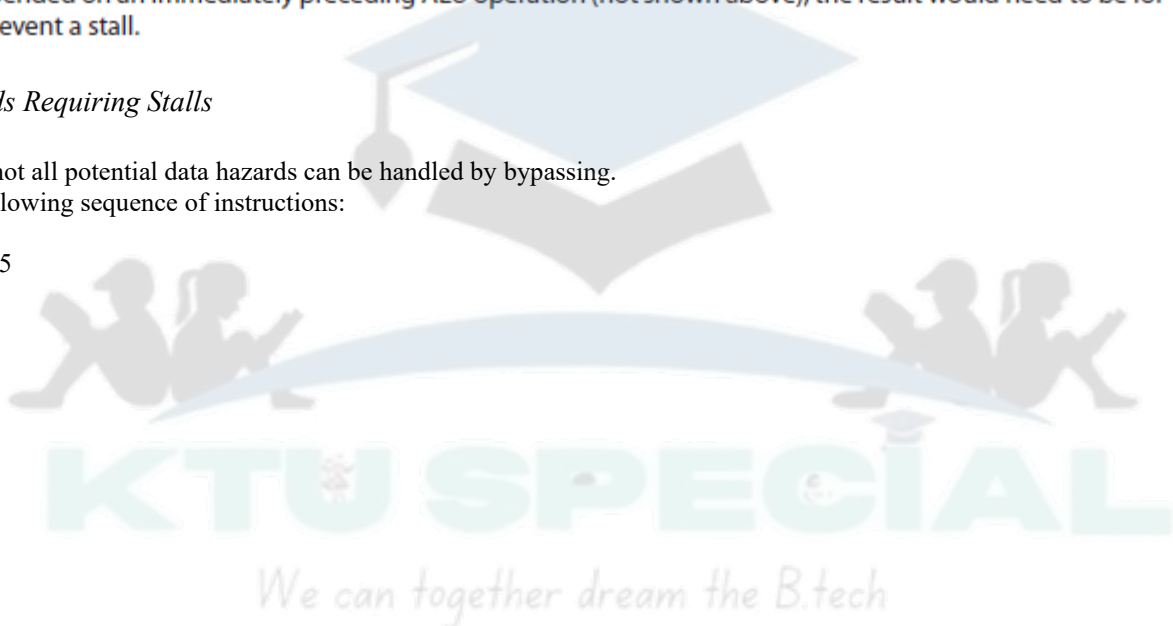
Figure C.8 Forwarding of operand required by stores during MEM. The result of the load is forwarded from the memory output to the memory input to be stored. In addition, the ALU output is forwarded to the ALU input for the address calculation of both the load and the store (this is no different than forwarding to another ALU operation). If the store depended on an immediately preceding ALU operation (not shown above), the result would need to be forwarded to prevent a stall.

2.Data Hazards Requiring Stalls

Unfortunately, not all potential data hazards can be handled by bypassing.

Consider the following sequence of instructions:

```
LD R1,0(R2)
DSUB R4,R1,R5
AND R6,R1,R7
OR R8,R1,R9
```



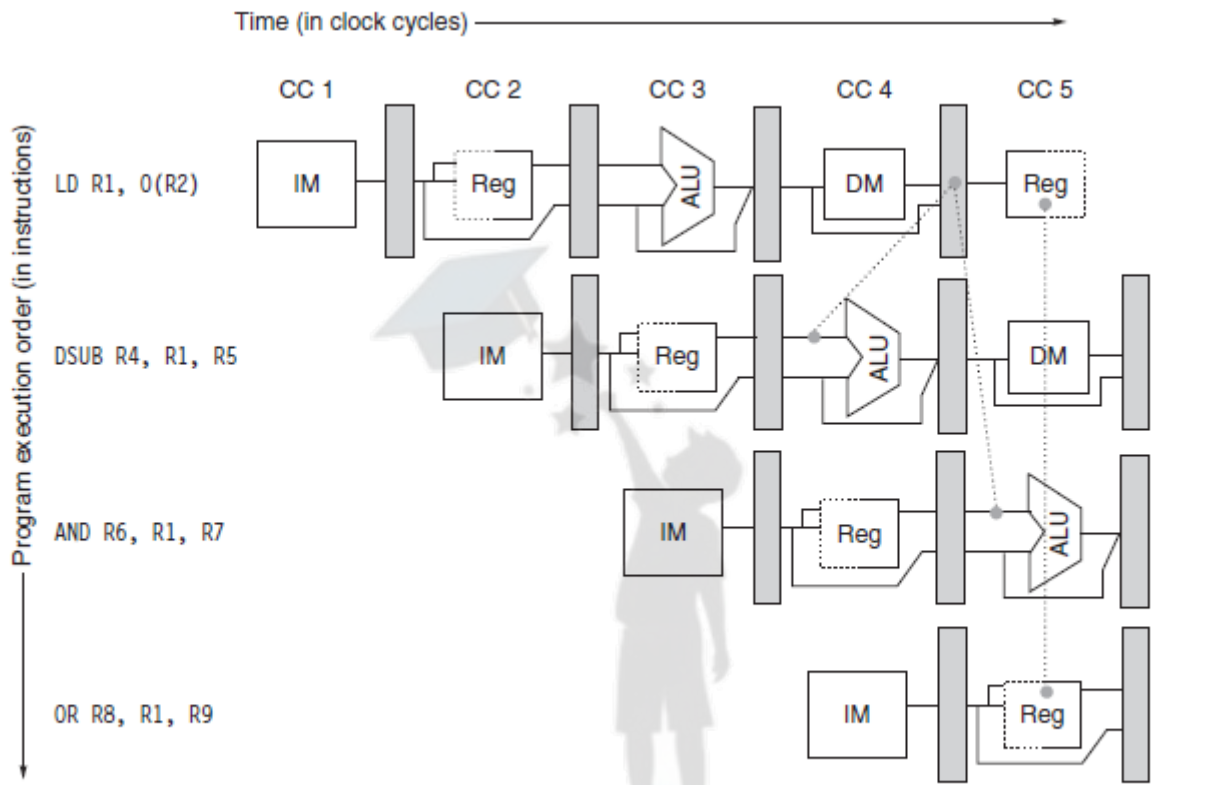


Figure C.9 The load instruction can bypass its results to the AND and OR instructions, but not to the DSUB, since that would mean forwarding the result in “negative time.”

Figure C.10 shows the pipeline before and after the stall using the names of the pipeline stages.

LD	R1,0(R2)	IF	ID	EX	MEM	WB				
DSUB	R4,R1,R5		IF	ID	EX	MEM	WB			
AND	R6,R1,R7			IF	ID	EX	MEM	WB		
OR	R8,R1,R9				IF	ID	EX	MEM	WB	
LD	R1,0(R2)	IF	ID	EX	MEM	WB				
DSUB	R4,R1,R5		IF	ID	stall	EX	MEM	WB		
AND	R6,R1,R7			IF	stall	ID	EX	MEM	WB	
OR	R8,R1,R9				stall	IF	ID	EX	MEM	WB

Figure C.10 In the top half, we can see why a stall is needed: The MEM cycle of the load produces a value that is needed in the EX cycle of the DSUB, which occurs at the same time. This problem is solved by inserting a stall, as shown in the bottom half.

Control/ Branch Hazards:

- Occur due to branch instructions that change the flow of execution.
- Example: The pipeline fetches the wrong instruction because the branch target is not yet known.

Control hazards can cause a greater performance loss for our MIPS pipeline than do data hazards. When a branch is executed, it may or may not change the PC to something other than its current value plus 4. Recall that if a branch changes the PC to its target address, it is a taken branch; if it falls through, it is not taken, or untaken. If instruction *i* is a taken branch, then the PC is normally not changed until the end of ID, after the completion of the address calculation and comparison.

Branch instruction	IF	ID	EX	MEM	WB		
Branch successor		IF	IF	ID	EX	MEM	WB
Branch successor + 1				IF	ID	EX	MEM
Branch successor + 2					IF	ID	EX

Figure C.11 A branch causes a one-cycle stall in the five-stage pipeline. The instruction after the branch is fetched, but the instruction is ignored, and the fetch is restarted once the branch target is known. It is probably obvious that if the branch is not taken, the second IF for branch successor is redundant. This will be addressed shortly.

Figure C.11 shows that the simplest method of dealing with branches is to redo the fetch of the instruction following a branch, once we detect the branch during ID (when instructions are decoded). The **first IF cycle is essentially a stall**, because it never performs useful work. You may have noticed that if the branch is untaken, then the repetition of the IF stage is unnecessary

Reducing Pipeline Branch Penalties/Control Hazards:

4 solutions to minimize the branch penalty for a branch which is static—(they are fixed for each branch during the entire execution). The time lost as a result of a branch instruction is called **branch penalty**.

The **simplest scheme** to handle branches **is to freeze or flush the pipeline**, holding or deleting any instructions after the branch until the branch destination is known. The attractiveness of this solution lies primarily in its simplicity both for hardware and software. It is the solution used earlier in the pipeline shown in Figure C.11. In this case, the branch penalty is fixed and cannot be reduced by software.

A higher-performance, and only slightly more complex, **scheme is to treat every branch as not taken**, simply allowing the hardware to continue as if the branch were not executed. Here, care must be taken not to change the processor state until the branch outcome is definitely known. The complexity of this scheme arises from having to know when the state might be changed by an instruction and how to “back out” such a change.

In the simple five-stage pipeline, this **predicted-not-taken** or predicted untaken scheme is implemented by continuing to fetch instructions as if the branch were a normal instruction.

If the branch is taken, however, we need to turn the fetched instruction into a no-op and restart the fetch at the target address. Figure C.12 shows both situations.



Untaken branch instruction	IF	ID	EX	MEM	WB				
Instruction $i + 1$		IF	ID	EX	MEM	WB			
Instruction $i + 2$			IF	ID	EX	MEM	WB		
Instruction $i + 3$				IF	ID	EX	MEM	WB	
Instruction $i + 4$					IF	ID	EX	MEM	WB
Taken branch instruction	IF	ID	EX	MEM	WB				
Instruction $i + 1$		IF	idle	idle	idle	idle			
Branch target			IF	ID	EX	MEM	WB		
Branch target + 1				IF	ID	EX	MEM	WB	
Branch target + 2					IF	ID	EX	MEM	WB

Figure C.12 The predicted-not-taken scheme and the pipeline sequence when the branch is untaken (top) and taken (bottom). When the branch is untaken, determined during ID, we fetch the fall-through and just continue. If the branch is taken during ID, we restart the fetch at the branch target. This causes all instructions following the branch to stall 1 clock cycle.

An **alternative scheme** is to **treat every branch as taken**. As soon as the branch is decoded and the target address is computed, we assume the branch to be taken and begin fetching and executing at the target. Because in our five-stage pipeline we don't know the target address any earlier than we know the branch outcome, there is **no advantage in this approach** for this pipeline. In some processors—especially those with implicitly set condition codes or more powerful (and hence slower) branch conditions—the branch target is known before the branch outcome, and a predicted-taken scheme might make sense. In either a predicted taken or predicted-not-taken scheme, the compiler can improve performance by **organizing the code** so that the most frequent path matches the hardware's choice.

A **fourth scheme** in use in some processors is called **delayed branch**. This technique was heavily used in early RISC processors and works reasonably well in the five-stage pipeline.

branch instruction
sequential successor
branch target if taken

The **sequential successor is in the branch delay slot**. This instruction is executed whether or not the branch is taken. The pipeline behavior of the five-stage pipeline with a branch delay is shown in **Figure C.13**.

We can together dream the B.tech



Untaken branch instruction	IF	ID	EX	MEM	WB		
Branch delay instruction ($i + 1$)		IF	ID	EX	MEM	WB	
Instruction $i + 2$			IF	ID	EX	MEM	WB
Instruction $i + 3$				IF	ID	EX	MEM
Instruction $i + 4$					IF	ID	EX
						MEM	WB
Taken branch instruction	IF	ID	EX	MEM	WB		
Branch delay instruction ($i + 1$)		IF	ID	EX	MEM	WB	
Branch target			IF	ID	EX	MEM	WB
Branch target + 1				IF	ID	EX	MEM
Branch target + 2					IF	ID	EX
						MEM	WB

Figure C.13 The behavior of a delayed branch is the same whether or not the branch is taken. The instructions in the delay slot (there is only one delay slot for MIPS) are executed. If the branch is untaken, execution continues with the instruction after the branch delay instruction; if the branch is taken, execution continues at the branch target. When the instruction in the branch delay slot is also a branch, the meaning is unclear: If the branch is not taken, what should happen to the branch in the branch delay slot? Because of this confusion, architectures with delay branches often disallow putting a branch in the delay slot.

Cases how to assign Delay Slot:



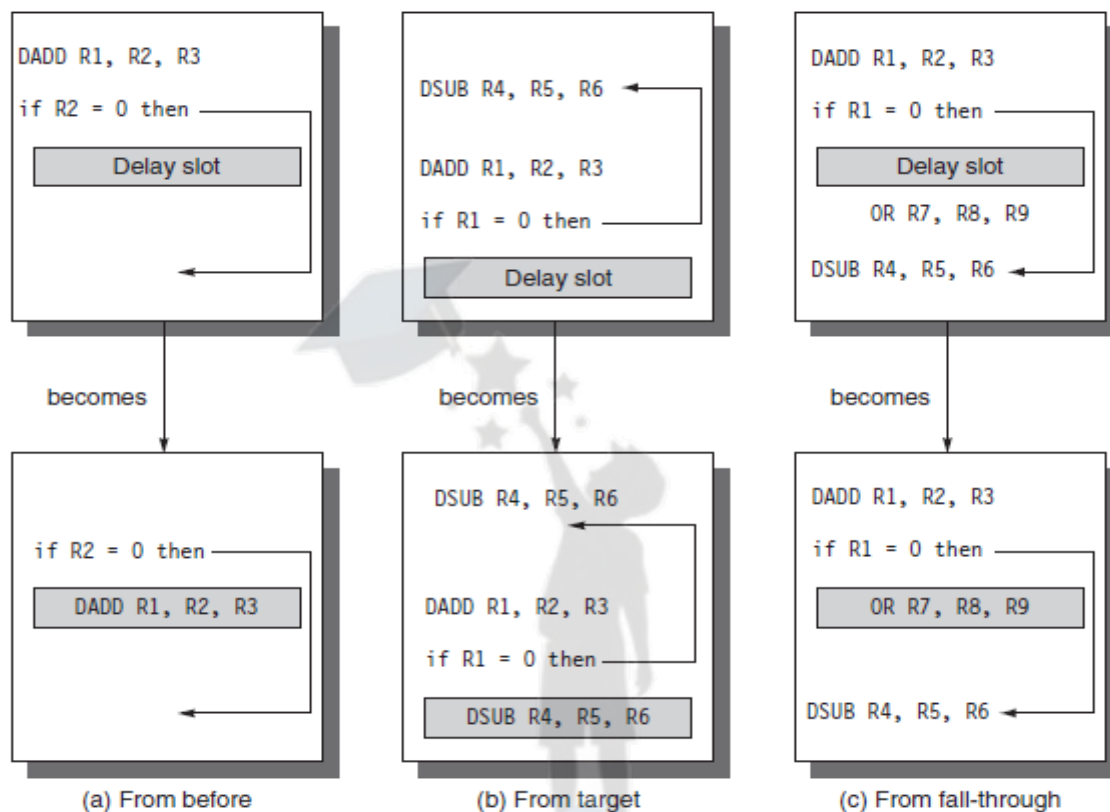


Figure C.14 Scheduling the branch delay slot. The top box in each pair shows the code before scheduling; the bottom box shows the scheduled code. In (a), the delay slot is scheduled with an independent instruction from before the branch. This is the best choice. Strategies (b) and (c) are used when (a) is not possible. In the code sequences for (b) and (c), the use of R1 in the branch condition prevents the DADD instruction (whose destination is R1) from being moved after the branch. In (b), the branch delay slot is scheduled from the target of the branch; usually the target instruction will need to be copied because it can be reached by another path. Strategy (b) is preferred when the branch is taken with high probability, such as a loop branch. Finally, the branch may be scheduled from the not-taken fall-through as in (c). To make this optimization legal for (b) or (c), it must be OK to execute the moved instruction when the branch goes in the unexpected direction. By OK we mean that the work is wasted, but the program will still execute correctly. This is the case, for example, in (c) if R7 were an unused temporary register when the branch goes in the unexpected direction.

The limitations on delayed-branch scheduling arise from: (1) the restrictions on the instructions that are scheduled into the delay slots, and (2) our ability to predict at compile time whether a branch is likely to be taken or not. To improve the ability of the compiler to fill branch delay slots, most processors with conditional branches have introduced a *canceling* or *nullifying* branch. In a canceling branch, the instruction includes the direction that the branch was predicted. When the branch behaves as predicted, the instruction in the branch delay slot is simply executed as it would normally be with a delayed branch. When the branch is incorrectly predicted, the instruction in the branch delay slot is simply turned into a no-op.

Reducing Pipeline Branch Penalties/Control Hazards:

- **Branch Prediction:** Predict the outcome of a branch and speculatively execute instructions.
- **Delayed Branch:** Reorganize instructions so that useful work is done in the branch delay slot.

Advanced Pipelining Techniques:

- **Superscalar Pipelines:** Multiple instructions are issued and executed in parallel.
- **Dynamic Scheduling:** Instructions are executed out of order to reduce stalls.
- **Speculative Execution:** Instructions are executed speculatively based on branch predictions.

Discuss about structural hazard and control hazard that occur in RISC instruction pipeline.

Discuss various types data hazards in a RISC instruction pipeline with appropriate examples.

Recap: Pipeline Hazards

Pipeline hazards are situations that prevent the next instruction in the pipeline from executing during its designated clock cycle. These hazards reduce the performance of a pipelined processor by causing stalls or delays. There are three main types of pipeline hazards: **structural hazards**, **data hazards**, and **control hazards**.

1. Structural Hazards

- **Definition:** Structural hazards occur when hardware resources are insufficient to support all possible combinations of instructions in the pipeline.
- **Cause:**
 - Resource conflicts arise when multiple instructions require the same hardware resource simultaneously.
 - Example: A single memory unit being accessed for both instruction fetch and data access in the same cycle.
- **Impact:**
 - The pipeline stalls, introducing bubbles (idle cycles) until the resource becomes available.
- **Solution:**
 - **Resource Duplication:** Add more hardware resources (e.g., separate instruction and data memory- as in **Harvard architecture**).
 - **Pipeline Interlocks:** Stall the pipeline until the resource conflict is resolved.
- **Example:**
 - A load instruction accesses memory for data while another instruction fetches the next instruction from memory, causing a conflict if there is only one memory unit.

2. Data Hazards

- **Definition:** Data hazards occur when an instruction depends on the result of a previous instruction that has not yet completed.
- **Types:**
 - **RAW (Read After Write): — True Dependency**
- Occurs when an instruction needs to read a register that a previous instruction is still writing to. **Most common** data hazard.
 - ☑ **Example:**
 1. ADD R1, R2, R3 ; $R1 = R2 + R3$
 2. SUB R4, R1, R5 ; $R4 = R1 - R5$ ← depends on R1 from line 1

- **Hazard:** Instruction 2 reads R1 before it has been updated by Instruction 1

- Solution: Use **forwarding** (bypassing) or stall the pipeline.

WAR (Write After Read): — Anti-dependency

- Occurs when a later instruction writes to a register before an earlier instruction reads it.
- **Rare in RISC pipelines** (since registers are written in **Write-Back (WB)** stage, which is after Read).
 - ☑ **Example:**
 1. SUB R4, R1, R2 ; $R4 = R1 - R2$
 2. ADD R1, R3, R5 ; $R1 = R3 + R5$
- **Hazard:** If instruction 2 writes to R1 before instruction 1 reads it.
 - Solution: Stall the pipeline to ensure the read happens first.
 - **WAW (Write After Write): — Output Dependency**
 - Occurs when two instructions write to the same register in an incorrect order.

- **Also rare in RISC**, due to in-order execution.

✓ *Example:*

1. ADD R1, R2, R3 ; $R1 = R2 + R3$
2. SUB R1, R4, R5 ; $R1 = R4 - R5$

- **Hazard:** If instruction 2 writes to R1 before instruction 1.
 - Solution: Stall the pipeline to ensure the writes occur in the correct order.

- **Impact:**
 - Data hazards can cause stalls, increasing the CPI (cycles per instruction).
- **Solution:**
 - **Forwarding (Bypassing):** Pass the result directly from one pipeline stage to another without waiting for it to be written back.
 - **Pipeline Interlocks:** Detect hazards and stall the pipeline until the data is available.

3. Control Hazards

- **Definition:** Control hazards occur when the pipeline encounters a branch or jump instruction that changes the program counter (PC), making it unclear which instruction to fetch next.
- **Cause:**
 - Branch instructions (e.g., conditional branches) delay the pipeline because the target address is not known until the branch condition is evaluated.
- **Impact:**
 - The pipeline may fetch the wrong instruction, leading to wasted cycles and stalls.
- **Solution:**
 - **Branch Prediction:**
 - Predict the outcome of the branch (taken or not taken) and speculatively fetch instructions.
 - If the prediction is correct, execution continues smoothly; if incorrect, the pipeline is flushed, and the correct instructions are fetched.
 - **Delayed Branch:**
 - Reorganize instructions so that useful work is done in the branch delay slot.
 - Example:
 - **Pipeline Interlocks:** Stall the pipeline until the branch condition is resolved.
- **Example:**
 - A branch instruction is encountered, but the target address is not known until the EX stage, causing a stall in the pipeline.

Summary Table

Hazard Type	Cause	Impact	Solution
Structural Hazards	Resource conflicts (e.g., single memory unit)	Pipeline stalls	Add resources, pipeline interlocks
Data Hazards	Dependencies between instructions	RAW, WAR, WAW stalls	Forwarding, pipeline interlocks
Control Hazards	Branch or jump instructions	Incorrect instruction fetch	Branch prediction, delayed branch

Q. Assume a pipelined processor with a single memory unit shared for both instruction fetch and data access. Show how a structural hazard arises when executing a LW followed by a SW, and propose an architectural solution to eliminate this hazard.

What's Happening?

- The processor has **only one memory unit**.
- This memory is used for both:
 - **Instruction Fetch (IF)**
 - **Data Access (Load/Store)**

This means **only one memory access can happen per cycle**. If two instructions try to access memory at the same time, a **conflict** (called a **structural hazard**) occurs.



Example Instruction Sequence:

1. LW R1, 0(R2) ; Load word from memory into R1
2. SW R3, 4(R4) ; Store word from R3 into memory



Timeline Diagram (With Structural Hazard)

Cycle	1	2	3	4	5
LW Stages	IF	ID	EX	MEM (data)	WB
SW Stages		IF	ID	EX	MEM (data)
Memory Use In each stage	Instruction memory	Instruction memory		✓ Data memory	✗ Conflict! (another instruction will need memory for fetch too)

👉 In **Cycle 5**, SW needs the **memory** to store data, but it's the **only memory unit**, and it's still being used (or just finishing) by the LW or needed by another instruction for **fetch**. This creates a **structural hazard**.



Solution: Harvard Architecture

To fix this:

- Use **two separate memory units**:
 - One for **instructions**
 - One for **data**

This allows:

- SW to access **data memory**
- While next instruction fetches from **instruction memory**

✓ Now both can work in the same cycle without conflict!



Fixed Timeline Diagram (Using Harvard Architecture)

Cycle	1	2	3	4	5
LW Stages	IF	ID	EX	MEM (data)	WB
SW Stages		IF	ID	EX	MEM (data)

Memory Use	Instruction memory	Instruction memory		 Data memory	Data memory + IF (no conflict)
-------------------	--------------------	--------------------	--	--	---------------------------------------

Consider the following instruction sequence:

BEQ R1, R2, L1

ADD R3, R4, R5

L1: SUB R6, R7, R8

Assume that the processor uses delayed branching with one delay slot. Reorder or modify the instruction sequence using delayed branching so that the pipeline efficiency is improved.

In **delayed branching**, the instruction immediately following a branch is **always executed**, whether or not the branch is taken. This instruction occupies the **branch delay slot**. To improve pipeline efficiency, we move a useful instruction into the delay slot—preferably one that doesn't affect the outcome of the branch.

◇ Original Sequence:

BEQ R1, R2, L1
 ADD R3, R4, R5 ← delay slot (inefficient if branch is taken)
 L1: SUB R6, R7, R8
 If BEQ is **taken**, ADD executes unnecessarily. This wastes a cycle.

◇ Improved Sequence with Delayed Branching:

We should move the instruction that **must execute regardless of the branch outcome** into the delay slot—if possible. Here, SUB is at the branch target and will be executed only **if the branch is taken**, so it **cannot** go into the delay slot. But ADD R3, R4, R5 is **not dependent** on the branch and can execute in both cases. So we can **move it after the branch target** and place it in the **delay slot**.

☒ Reordered Instruction Sequence:

BEQ R1, R2, L1
 ADD R3, R4, R5 ← moved to delay slot
 L1: SUB R6, R7, R8

◇ Result:

- If BEQ is **not taken** → ADD executes as usual.
- If BEQ is **taken** → ADD still executes **before jumping**, but it's useful, so no cycle is wasted.
- This improves pipeline efficiency without altering program behavior.
-   In the **original**, the ADD may have been placed there *naively*.
-   In the **optimized**, ADD is *deliberately chosen* for the delay slot to improve efficiency.

A Simple Implementation of MIPS

Every MIPS instruction can be implemented in at most 5 clock cycles. The 5 clock cycles are as follows:

1. Instruction fetch cycle (IF): [download from ktuspecial.in](https://www.ktuspecial.in)

$IR \leftarrow \text{Mem}[PC];$

$NPC \leftarrow PC + 4;$

Operation—Send out the PC and fetch the instruction from memory into the instruction register (IR); increment the PC by 4 to address the next sequential instruction. The IR is used to hold the instruction that will be needed on subsequent clock cycles; likewise, the register NPC is used to hold the next sequential PC.

2. Instruction decode/register fetch cycle (ID):

$A \leftarrow \text{Regs}[rs];$

$B \leftarrow \text{Regs}[rt];$

$\text{Imm} \leftarrow \text{sign-extended immediate field of IR};$

Operation—Decode the instruction and access the register file to read the registers (rs and rt are the register specifiers). The outputs of the generalpurpose registers are read into two temporary registers (A and B) for use in later clock cycles. The lower 16 bits of the IR are also sign extended and stored into the temporary register Imm, for use in the next cycle.

Decoding is done in parallel with reading registers, which is possible because these fields are at a fixed location in the MIPS instruction format. Because the immediate portion of an instruction is located in an identical place in every MIPS format, the sign-extended immediate is also calculated during this cycle in case it is needed in the next cycle.

3. Execution/effective address cycle (EX):

The ALU operates on the operands prepared in the prior cycle, performing one of four functions depending on the MIPS instruction type:

■ Memory reference:

$\text{ALUOutput} \leftarrow A + \text{Imm};$

Operation—The ALU adds the operands to form the effective address and places the result into the register ALUOutput.

■ Register-register ALU instruction:

$\text{ALUOutput} \leftarrow A \text{ func } B;$

Operation—The ALU performs the operation specified by the function code on the value in register A and on the value in register B. The result is placed in the temporary register ALUOutput.

■ Register-Immediate ALU instruction:

$\text{ALUOutput} \leftarrow A \text{ op } \text{Imm};$

Operation—The ALU performs the operation specified by the opcode on the value in register A and on the value in register Imm. The result is placed in the temporary register ALUOutput.

■ Branch:

$\text{ALUOutput} \leftarrow NPC + (\text{Imm} \ll 2);$

$\text{Cond} \leftarrow (A == 0)$

Operation—The ALU adds the NPC to the sign-extended immediate value in Imm, which is shifted left by 2 bits to create a word offset, to compute the address of the branch target. Register A, which has been read in the prior cycle, is checked to determine whether the branch is taken. Since we are considering only one form of branch (BEQZ), the comparison is against 0. Note that BEQZ is actually a pseudoinstruction that translates to a BEQ with R0 as an operand. For simplicity, this is the only form of branch we consider. The load-store architecture of MIPS means that effective address and execution cycles can be combined into a single clock cycle, since no instruction needs to simultaneously calculate a data address, calculate an instruction target address, and perform an operation on the data. The other integer instructions not included above are jumps of various forms, which are similar to branches.

4. Memory access/branch completion cycle (MEM):

The PC is updated for all instructions: $PC \leftarrow NPC$;

- Memory reference:

$LMD \leftarrow Mem[ALUOutput]$ or

$Mem[ALUOutput] \leftarrow B$;

Operation—Access memory if needed. If instruction is a load, data return from memory and are placed in the LMD (load memory data) register; if it is a store, then the data from the B register are written into memory. In either case, the address used is the one computed during the prior cycle and stored in the register ALUOutput.

- Branch:

if (cond) $PC \leftarrow ALUOutput$

Operation—If the instruction branches, the PC is replaced with the branch destination address in the register ALUOutput.

5. Write-back cycle (WB):

- Register-register ALU instruction:

$Regs[rd] \leftarrow ALUOutput$;

- Register-immediate ALU instruction:

$Regs[rt] \leftarrow ALUOutput$;

- Load instruction:

$Regs[rt] \leftarrow LMD$;

Operation—Write the result into the register file, whether it comes from the memory system (which is in LMD) or from the ALU (which is in ALUOutput); the register destination field is also in one of two positions (rd or rt) depending on the effective opcode.

Figure C.21 shows how an instruction flows through the data path.



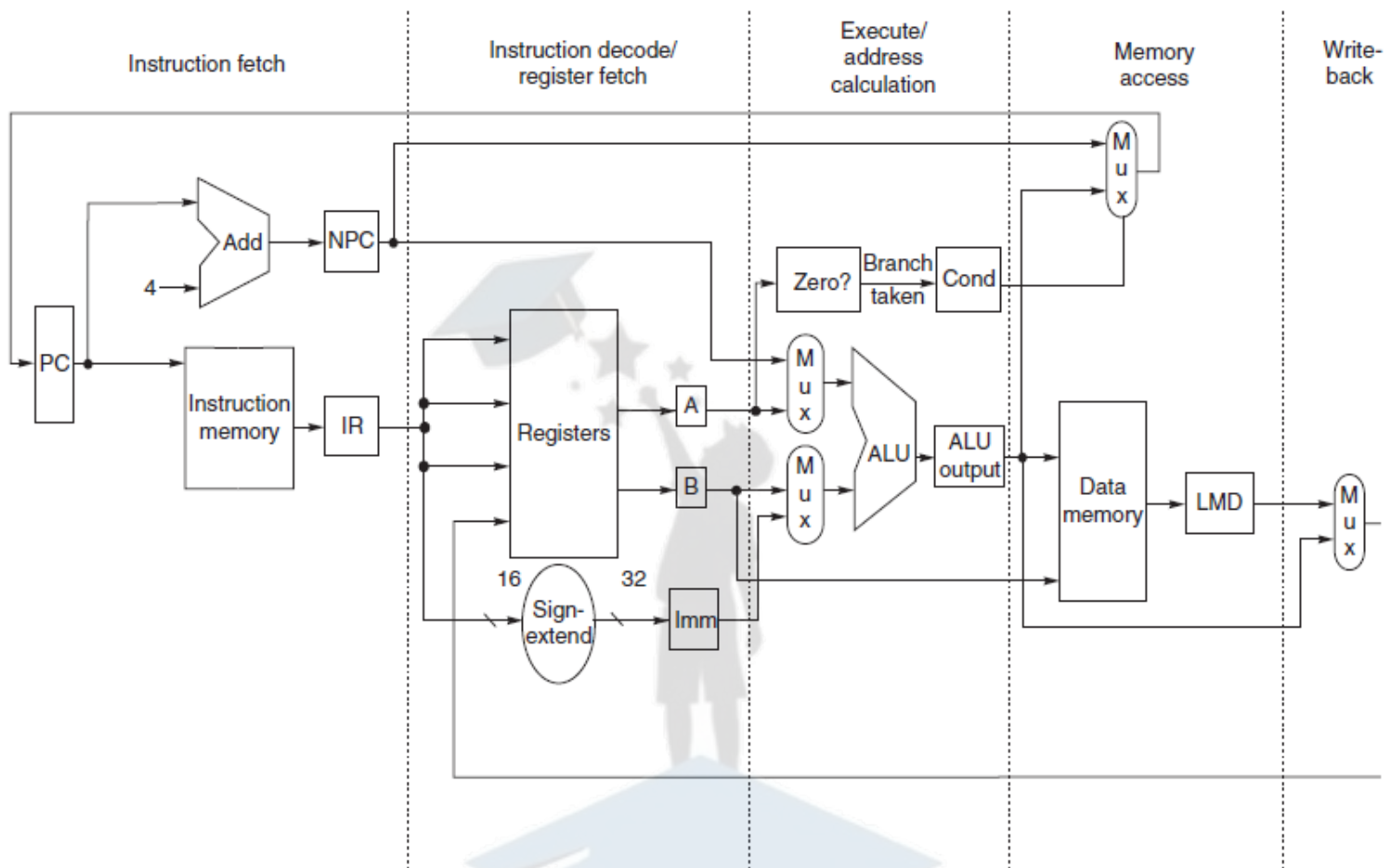


Figure C.21 The implementation of the MIPS data path allows every instruction to be executed in 4 or 5 clock cycles. Although the PC is shown in the portion of the data path that is used in instruction fetch and the registers are shown in the portion of the data path that is used in instruction decode/register fetch, both of these functional units are read as well as written by an instruction. Although we show these functional units in the cycle corresponding to where they are read, the PC is written during the memory access clock cycle and the registers are written during the write-back clock cycle. In both cases, the writes in later pipe stages are indicated by the multiplexer output (in memory access or write-back), which carries a value back to the PC or registers. These backward-flowing signals introduce much of the complexity of pipelining, since they indicate the possibility of hazards.

In such cases where the CPU is implemented, so that every instruction takes 1 long clock cycle, the temporary registers would be deleted, since there would not be any communication across clock cycles within an instruction. Every instruction would execute in 1 long clock cycle, writing the result into the data memory, registers, or PC at the end of the clock cycle. The CPI would be one for such a processor. The clock cycle, however, would be roughly equal to five times the clock cycle of the multicycle processor, since every instruction would need to traverse all the functional units.

Pipelining the data path requires that values passed from one pipe stage to the next must be placed in registers. Figure C.22 shows the MIPS pipeline with the appropriate registers, called pipeline registers or pipeline latches, between each pipeline stage. The registers are labeled with the names of the stages they connect. If we tried to just use the temporary registers we had in our earlier unpipelined data path, values could be overwritten before all uses were completed.

Pipeline registers are crucial. They store intermediate results between stages, preventing data overwriting and ensuring correct data flow between clock cycles. These registers are essential for the proper functioning of the pipeline and prevent interference between instructions in different stages.

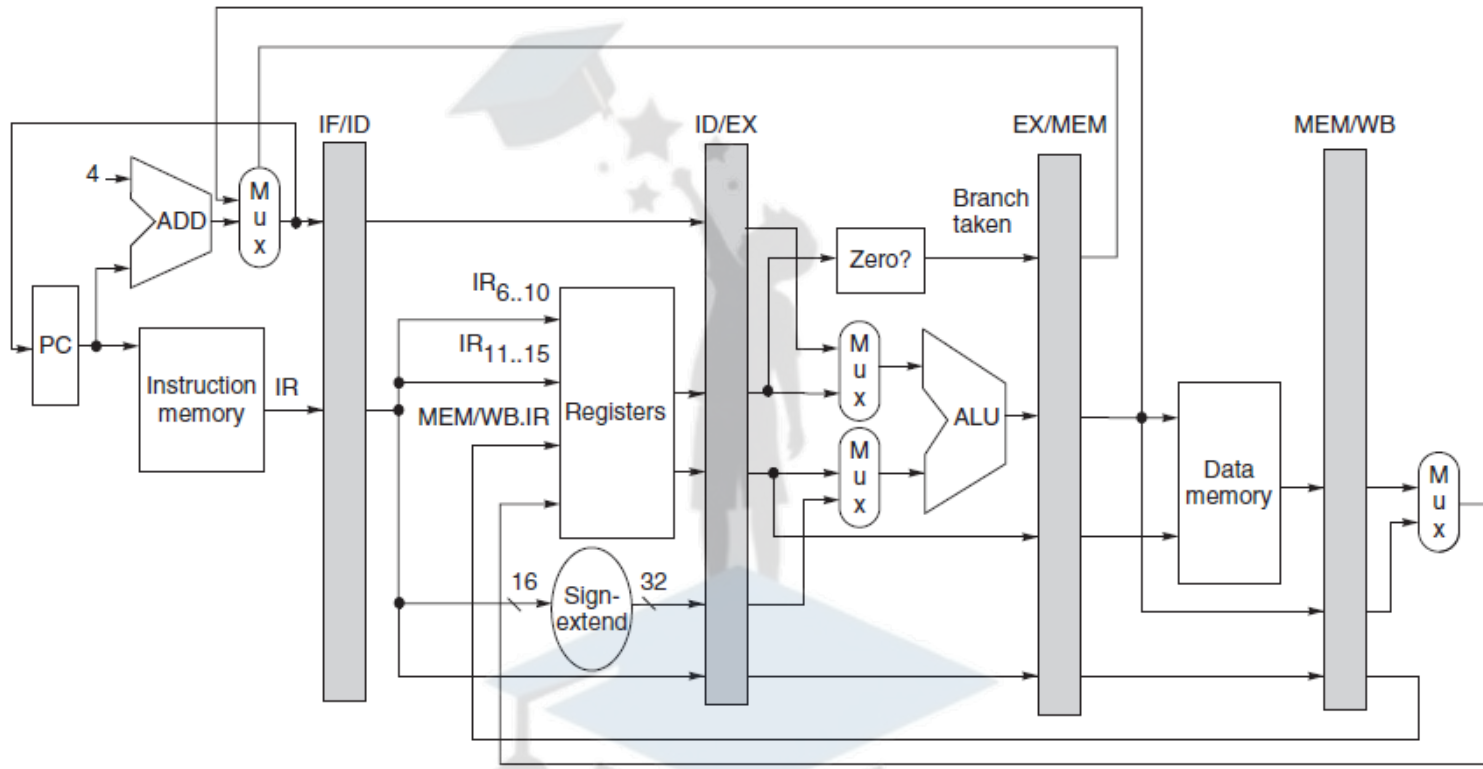


Figure C.22 The data path is pipelined by adding a set of registers, one between each pair of pipe stages. The registers serve to convey values and control information from one stage to the next. We can also think of the PC as a pipeline register, which sits before the IF stage of the pipeline, leading to one pipeline register for each pipe stage. Recall that the PC is an edge-triggered register written at the end of the clock cycle; hence, there is no race condition in writing the PC. The selection multiplexer for the PC has been moved so that the PC is written in exactly one stage (IF). If we didn't move it, there would be a conflict when a branch occurred, since two instructions would try to write different values into the PC. Most of the data paths flow from left to right, which is from earlier in time to later. The paths flowing from right to left (which carry the register write-back information and PC information on a branch) introduce complications into our pipeline.

To control this simple pipeline we need only determine how to set the control for the four multiplexers in the data path of [Figure C.22](#). The two multiplexers in the ALU stage are set depending on the instruction type, which is dictated by the IR field of the ID/EX register. The top ALU input multiplexer is set by whether the instruction is a branch or not, and the bottom multiplexer is set by whether the instruction is a register-register ALU operation or any other type of operation. The multiplexer in the IF stage chooses whether to use the value of the incremented PC or the value of the EX/MEM.ALUOutput (the branch target) to write into the PC. This multiplexer is controlled by the field EX/MEM.cond. The fourth multiplexer is controlled by whether the instruction in the WB stage is a load or an ALU operation. In addition to these four multiplexers, there is one additional multiplexer needed that is not drawn in [Figure C.22](#), but whose existence is clear from looking at the WB stage of an ALU operation. The destination register field is in one of two different places depending on the instruction type (register-register ALU versus either ALU immediate or load). Thus, we will need a multiplexer to choose the correct portion of the IR in the MEM/WB register to specify the register destination field, assuming the instruction writes a register.

Implementing the Control for the MIPS Pipeline

The process of letting an instruction move from the instruction decode stage (ID) into the execution stage (EX) of this pipeline is usually called *instruction issue*; an instruction that has made this step is said to have *issued*. For the MIPS integer pipeline, all the data hazards can be checked during the ID phase of the pipeline.

If a data hazard exists, the instruction is stalled before it is issued. Figure C.24 lists the variety of circumstances that we must handle.

Detecting interlocks early in the pipeline reduces the hardware Complexity

Situation	Example code sequence	Action
No dependence	LD R1, 45(R2) DADD R5, R6, R7 DSUB R8, R6, R7 OR R9, R6, R7	No hazard possible because no dependence exists on R1 in the immediately following three instructions.
Dependence requiring stall	LD R1, 45(R2) DADD R5, R1, R7 DSUB R8, R6, R7 OR R9, R6, R7	Comparators detect the use of R1 in the DADD and stall the DADD (and DSUB and OR) before the DADD begins EX.
Dependence overcome by forwarding	LD R1, 45(R2) DADD R5, R6, R7 DSUB R8, R1, R7 OR R9, R6, R7	Comparators detect use of R1 in DSUB and forward result of load to ALU in time for DSUB to begin EX.
Dependence with accesses in order	LD R1, 45(R2) DADD R5, R6, R7 DSUB R8, R6, R7 OR R9, R1, R7	No action required because the read of R1 by OR occurs in the second half of the ID phase, while the write of the loaded data occurred in the first half.

Figure C.24 Situations that the pipeline hazard detection hardware can see by comparing the destination and sources of adjacent instructions. This table indicates that the only comparison needed is between the destination and the sources on the two instructions following the instruction that wrote the destination. In the case of a stall, the pipeline dependences will look like the third case once execution continues. Of course, hazards that involve R0 can be ignored since the register always contains 0, and the test above could be extended to do this.

Dealing with Branches in the Pipeline

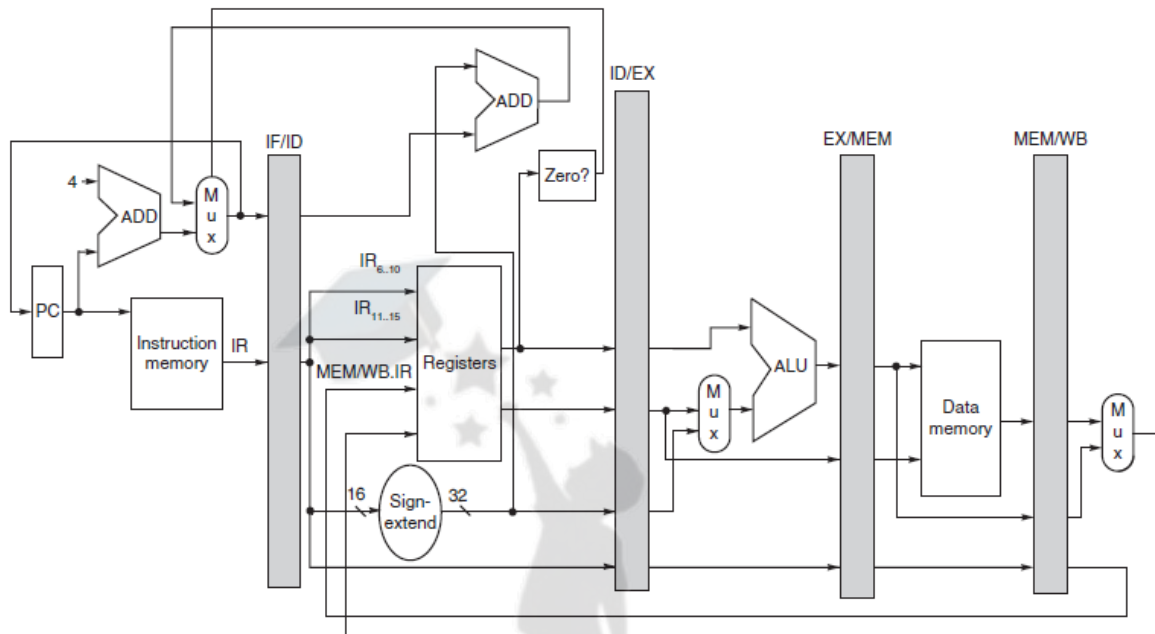


Figure C.28 The stall from branch hazards can be reduced by moving the zero test and branch-target calculation into the ID phase of the pipeline. Notice that we have made two important changes, each of which removes 1 cycle from the 3-cycle stall for branches. The first change is to move both the branch-target address calculation and the branch condition decision to the ID cycle. The second change is to write the PC of the instruction in the IF phase, using either the branch-target address computed during ID or the incremented PC computed during IF. In comparison, Figure C.22 obtained the branch-target address from the EX/MEM register and wrote the result during the MEM clock cycle. As mentioned in Figure C.22, the PC can be thought of as a pipeline register (e.g., as part of ID/IF), which is written with the address of the next instruction at the end of each IF cycle.

What Makes Pipelining Hard to Implement?

Dealing with Exceptions

Exceptional situations are harder to handle in a pipelined CPU because the overlapping of instructions makes it more difficult to know whether an instruction can safely change the state of the CPU.

Types of Exceptions and Requirements

The terminology used to describe exceptional situations where the normal execution order of instruction is changed varies among CPUs. The terms *interrupt*, *fault*, and *exception* are used, although not in a consistent fashion. We use the term *exception* to cover all these mechanisms, including the following:

- I/O device request
- Invoking an operating system service from a user program
- Tracing instruction execution
- Breakpoint (programmer-requested interrupt)
- Integer arithmetic overflow
- FP arithmetic anomaly
- Page fault (not in main memory)
- Misaligned memory accesses (if alignment is required)
- Memory protection violation
- Using an undefined or unimplemented instruction
- Hardware malfunctions
- Power failure

The requirements on exceptions can be characterized on five semi independent axes:

1. *Synchronous versus asynchronous*—If the event occurs at the same place every time the program is executed with the same data and memory allocation, the event is *synchronous*. With the exception of hardware malfunctions, *asynchronous* events are caused by devices external to the CPU and memory. Asynchronous events usually can be handled after the completion of the current instruction, which makes them easier to handle.
2. *User requested versus coerced*—If the user task directly asks for it, it is a *user-requested* event. In some sense, user-requested exceptions are not really exceptions, since they are predictable. They are treated as exceptions, however, because the same mechanisms that are used to save and restore the state are used for these user-requested events. Because the only function of an instruction that triggers this exception is to cause the exception, user requested exceptions can always be handled after the instruction has completed. *Coerced* exceptions are caused by some hardware event that is not under the control of the user program. Coerced exceptions are harder to implement because they are not predictable.
3. *User maskable versus user nonmaskable*—If an event can be masked or disabled by a user task, it is *user maskable*. This mask simply controls whether the hardware responds to the exception or not.
4. *Within versus between instructions*—This classification depends on whether the event prevents instruction completion by occurring in the middle of execution—no matter how short—or whether it is recognized *between* instructions. Exceptions that occur *within* instructions are usually synchronous, since the instruction triggers the exception. It's harder to implement exceptions that occur within instructions than those between instructions, since the instruction must be stopped and restarted. Asynchronous exceptions that occur within instructions arise from catastrophic situations (e.g., hardware malfunction) and always cause program termination.
5. *Resume versus terminate*—If the program's execution always stops after the interrupt, it is a *terminating* event. If the program's execution continues after the interrupt, it is a *resuming* event. It is easier to implement exceptions that terminate execution, since the CPU need not be able to restart execution of the same program after handling the exception.

KTU SPECIAL

We can together dream the B.tech



APJ ABDUL KALAM
TECHNOLOGICAL UNIVERSITY

मनसुखमिह, उद्यममिह, मनुजसुखमिह

Exception type	Synchronous vs. asynchronous	User request vs. coerced	User maskable vs. nonmaskable	Within vs. between instructions	Resume vs. terminate
I/O device request	Asynchronous	Coerced	Nonmaskable	Between	Resume
Invoke operating system	Synchronous	User request	Nonmaskable	Between	Resume
Tracing instruction execution	Synchronous	User request	User maskable	Between	Resume
Breakpoint	Synchronous	User request	User maskable	Between	Resume
Integer arithmetic overflow	Synchronous	Coerced	User maskable	Within	Resume
Floating-point arithmetic overflow or underflow	Synchronous	Coerced	User maskable	Within	Resume
Page fault	Synchronous	Coerced	Nonmaskable	Within	Resume
Misaligned memory accesses	Synchronous	Coerced	User maskable	Within	Resume
Memory protection violations	Synchronous	Coerced	Nonmaskable	Within	Resume
Using undefined instructions	Synchronous	Coerced	Nonmaskable	Within	Terminate
Hardware malfunctions	Asynchronous	Coerced	Nonmaskable	Within	Terminate
Power failure	Asynchronous	Coerced	Nonmaskable	Within	Terminate

Figure C.31 Five categories are used to define what actions are needed for the different exception types shown

Pipeline stage	Problem exceptions occurring
IF	Page fault on instruction fetch; misaligned memory access; memory protection violation
ID	Undefined or illegal opcode
EX	Arithmetic exception
MEM	Page fault on data fetch; misaligned memory access; memory protection violation
WB	None

Figure C.32 Exceptions that may occur in the MIPS pipeline. Exceptions raised from instruction or data memory access account for six out of eight cases.

Extending the MIPS Pipeline to Handle Multicycle Operations

Handling Multicycle Operations

Multicycle operations, which require more than one clock cycle in the EX stage, pose a unique challenge. To handle multicycle operations in a RISC-V integer pipeline, like a MIPS pipeline in the context of MIPS floating-point operations, you might:

* Extend the EX stage: Divide the EX stage into multiple sub-stages, each taking one clock cycle. This increases the pipeline's depth but allows for longer operations.

- * Add additional functional units: If the multicycle operation is computationally intensive (e.g., division), adding a dedicated functional unit can prevent it from blocking other instructions.
- * Implement stalling and forwarding: Stalling mechanisms are needed to handle data hazards involving multicycle operations. Forwarding can help reduce the number of stalls.
- * Careful scheduling: The compiler or hardware scheduler must carefully schedule instructions to minimize hazards and maximize pipeline efficiency.

This is easier to grasp if we imagine the FP instructions as having the same pipeline as the integer instructions, with two important changes. First, the EX cycle may be repeated as many times as needed to complete the operation—the number of repetitions can vary for different operations. Second, there may be multiple FP functional units. A stall will occur if the instruction to be issued will cause either a structural hazard for the functional unit it uses or a data hazard.

For this section, let's assume that there are four separate functional units in our MIPS implementation:

1. The main integer unit that handles loads and stores, integer ALU operations, and branches
2. FP and integer multiplier
3. FP adder that handles FP add, subtract, and conversion
4. FP and integer divider

If we also assume that the execution stages of these functional units are not pipelined, then Figure C.33 shows the resulting pipeline structure. Because EX is not pipelined, no other instruction using that functional unit may issue until the previous instruction leaves EX. Moreover, if an instruction cannot proceed to the EX stage, the entire pipeline behind that instruction will be stalled.

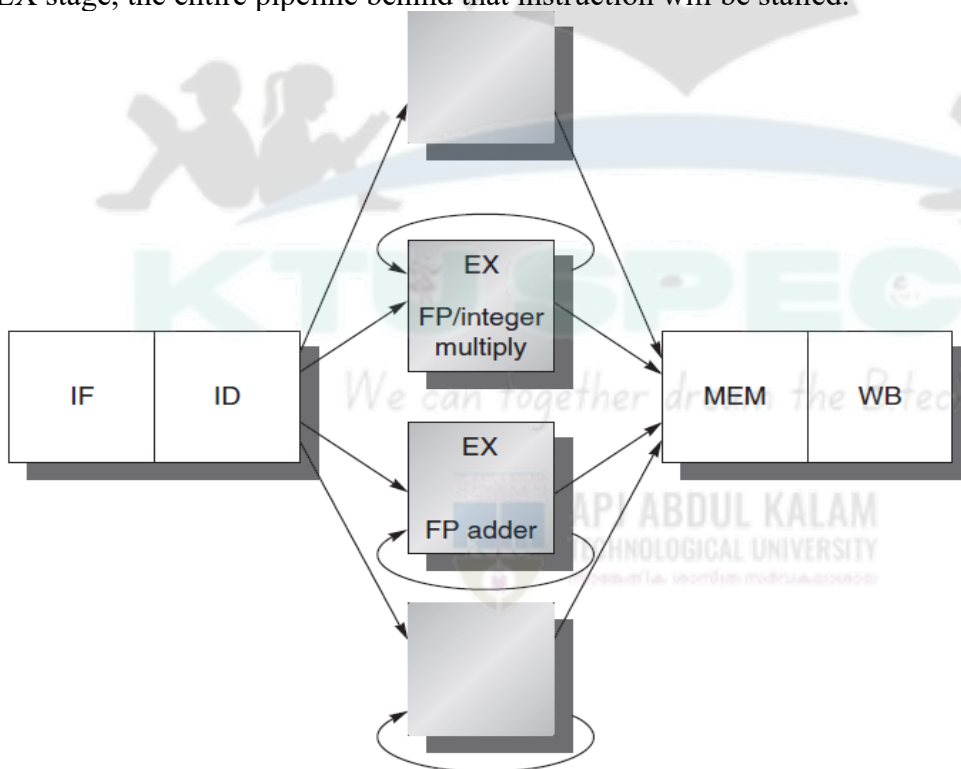


Figure C.33 The MIPS pipeline with three additional unpipelined, floating-point, functional units. Because only one instruction issues on every clock cycle, all instructions go through the standard pipeline for integer operations. The FP operations simply loop when they reach the EX stage. After they have finished the EX stage, they proceed to MEM and WB to complete execution.

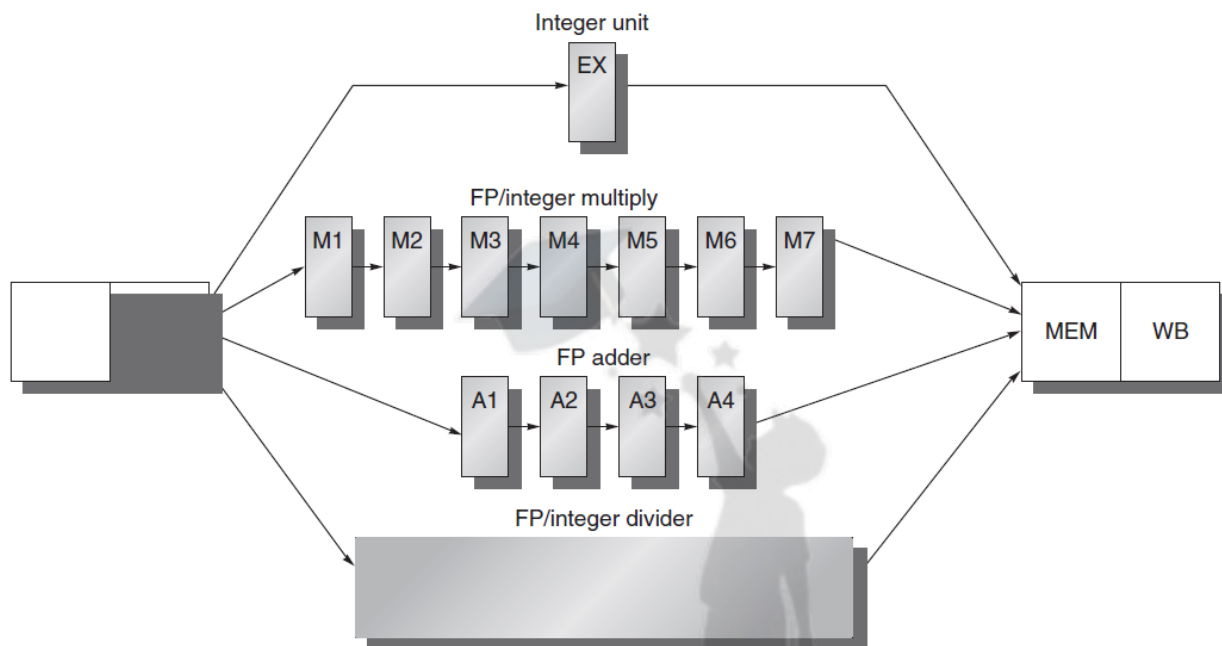


Figure C.35 A pipeline that supports multiple outstanding FP operations. The FP multiplier and adder are fully pipelined and have a depth of seven and four stages, respectively. The FP divider is not pipelined, but requires 24 clock cycles to complete. The latency in instructions between the issue of an FP operation and the use of the result of that operation without incurring a RAW stall is determined by the number of cycles spent in the execution stages. For example, the fourth instruction after an FP add can use the result of the FP add. For integer ALU operations, the depth of the execution pipeline is always one and the next instruction can use the results.

Hazards and Forwarding in Longer Latency Pipelines

There are a number of different aspects to the hazard detection and forwarding for a pipeline like that shown in Figure C.35.

1. Because the divide unit is not fully pipelined, structural hazards can occur. These will need to be detected and issuing instructions will need to be stalled.
2. Because the instructions have varying running times, the number of register writes required in a cycle can be larger than 1.
3. Write after write (WAW) hazards are possible, since instructions no longer reach WB in order. Note that write after read (WAR) hazards are not possible, since the register reads always occur in ID.
4. Instructions can complete in a different order than they were issued, causing problems with exceptions; we deal with this in the next subsection.
5. Because of longer latency of operations, stalls for RAW hazards will be more frequent.

Instruction	Clock cycle number																
	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
L.D F4,0(R2)	IF	ID	EX	MEM	WB												
MUL.D F0,F4,F6		IF	ID	Stall	M1	M2	M3	M4	M5	M6	M7	MEM	WB				
ADD.D F2,F0,F8			IF	Stall	ID	Stall	Stall	Stall	Stall	Stall	Stall	A1	A2	A3	A4	MEM	WB
S.D F2,0(R2)					IF	Stall	Stall	Stall	Stall	Stall	Stall	ID	EX	Stall	Stall	Stall	MEM

Figure C.37 A typical FP code sequence showing the stalls arising from RAW hazards. The longer pipeline substantially raises the frequency of stalls versus the shallower integer pipeline. Each instruction in this sequence is dependent on the previous and proceeds as soon as data are available, which assumes the pipeline has full bypassing and forwarding. The S.D must be stalled an extra cycle so that its MEM does not conflict with the ADD.D. Extra hardware could easily handle this case.

Assuming that the pipeline does all hazard detection in ID, there are three checks that must be performed before an instruction can issue:

1. Check for structural hazards—Wait until the required functional unit is not busy (this is only needed for divides in this pipeline) and make sure the register write port is available when it will be needed.
2. Check for a RAW data hazard—Wait until the source registers are not listed as pending destinations in a pipeline register that will not be available when this instruction needs the result. A number of checks must be made here, depending on both the source instruction, which determines when the result will be available, and the destination instruction, which determines when the value is needed. For example, if the instruction in ID is an FP operation with source register F2, then F2 cannot be listed as a destination in ID/A1, A1/A2, or A2/A3, which correspond to FP add instructions that will not be finished when the instruction in ID needs a result. (ID/A1 is the portion of the output register of ID that is sent to A1.) Divide is somewhat more tricky, if we want to allow the last few cycles of a divide to be overlapped, since we need to handle the case when a divide is close to finishing as special. In practice, designers might ignore this optimization in favor of a simpler issue test.
3. Check for a WAW data hazard—Determine if any instruction in A1, . . . , A4, D, M1, . . . , M7 has the same register destination as this instruction. If so, stall the issue of the instruction in ID.

Problems

A 5-stage pipelined processor has the stages: Instruction Fetch (IF), Instruction Decode (ID), Execute (EX), Memory access (MEM) and Write Back (WB). Each stage requires only one cycle. Operand forwarding is used in the pipeline. What is the number of clock cycles needed to execute the following sequence of instructions?

//ADD TWO INTEGER ARRAYS

LW R4, # 400

LW R1, 0 (R4) ; Load first operand

LW R2, 400 (R4) ; Load second operand

ADDI R3, R1, R2 ; Add operands

SW R3, 0 (R4) ; Store result

download from ktuspecial.in

SUB R4, R4, #4 ; Calculate address of next element

Instruction	1	2	3	4	5	6	7	8	9	10	11
1. LW R4, #400	IF	ID	EX	MEM	WB						
2. LW R1, 0(R4)		IF	ID	EX	MEM	WB					
3. LW R2, 400(R4)			IF	ID	EX	MEM	WB				
4. ADDI R3, R1, R2				ID	ID*	EX	MEM	WB			
5. SW R3, 0(R4)						IF	ID	EX	MEM	WB	
6. SUB R4, R4, #4							IF	ID	EX	MEM	WB

Total Clock Cycles = 11

ID is repeated at cycle 5 for ADDI to represent the 1-cycle stall due to a **load-use hazard** with R1 or R2.

Q. Consider the following sequence of instructions being processed on the pipelined 5-stage RISC processor.

Add R4, R2, R3

Store R5, #100(R4)

Load R6, #200(R4)

Subtract R7, R5, R6

Identify all the data dependencies in the above instruction sequence. For each dependency, indicate the two instructions and the register that causes the dependency.(6)

There are Four data dependencies in this instruction sequence:

- (i) Store instruction depends on Add instruction for register R4
- (ii) Load instruction depends on Add instruction for register R4
- (iii) Subtract instruction depends on Store instruction for register R5
- (iv) Subtract instruction depends on Load instruction for register R6

Identification of dependencies 2 marks each (3x2=6 marks)

1. A 5-stage pipelined processor has Instruction Fetch (IF), Instruction Decode (ID), Operand Fetch (OF), Perform Operation (PO) and Write Operand (WO) stages. The IF, ID, OF and WO stages take 1 clock cycle each for any instruction. The PO stage takes 1 clock cycle for ADD and SUB instructions, 3 clock cycles for MUL instruction, and 6 clock cycles for DIV instruction respectively. Operand forwarding is used in the pipeline. What is the number of clock cycles needed to execute the following sequence of instructions?

Instruction Meaning of instruction

I0 : MUL R2 ,R0 ,R1 $R2 = R0 * R1$

I1 : DIV R5 ,R3, R4 $R5 = R3 / R4$

I2 : ADD R2 ,R5 ,R2 $R2 = R5 + R2$

I3 : SUB R5 ,R2 ,R6 $R5 = R2 - R6$

It is given that there is **operand forwarding**. In the case of operand forwarding, the updated value from the previous instruction's PO stage is forwarded to the present instruction's PO stage. Here there's a RAW dependency between I1-I2 for R5 and between I2-I3 for R2. These dependencies are resolved by using operand forwarding as shown in the below table. The total number of clock cycles needed is 15.

CLOCK CYCLE	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
I0	IF	ID	OF	PO	PO	PO	WO								
I1		IF	ID	OF	OF	OF	PO	PO	PO	PO	PO	PO	WO		
I2			IF	ID	ID	ID	OF	OF	OF	OF	OF	OF	PO	WO	
I3				IF	IF	IF	ID	ID	ID	ID	ID	ID	OF	PO	WO

A5-stage pipelined processor has Instruction Fetch(IF), Instruction Decode (ID), Operand Fetch (OF), Perform Operation (PO) and Write Operand (WO) stages. The IF, ID, OF and WO stages take 1 clock cycle each for any instruction. The PO stage takes 1 clock cycle for ADD and SUB instructions, 4 clock cycles for MUL instruction, and 7 clock cycles for DIV instruction respectively.Operand forwarding is used in the pipeline. Obtain the number of clock cycles needed to execute the following sequence of instructions?

Instruction	Meaning of instruction
I1 :MUL R2 ,R0 ,R1	R2 = R0 *R1
I1 :DIV R5 ,R3 ,R4	R5 = R3/R4
I2 :ADD R2 ,R5 ,R2	R2 = R5+R2
I3 :SUB R5 ,R2 ,R6	R5 = R2-R6

Ans. 17 clock cycles

Q. Consider a pipelined processor with the following four stages- (7 marks)

- IF : Instruction Fetch**
- ID : Instruction Decode and Operand Fetch**
- EX : Execute**
- WB : Write Back**

The IF, ID and WB stages take one clock cycle each to complete the operation. The number of clock cycles for the EX stage depends on the instruction. The ADD and SUB instructions need 1 clock cycle and the MUL instruction need 3 clock cycles in the EX stage. Operand forwarding is used in the pipelined.

processor. What is the number of clock cycles taken to complete the following sequence of instructions?

ADD R2, R1, R0 { R2 $\leftarrow$ R0 + R1 }

MUL R4, R3, R2 { R4 $\leftarrow$ R3 x R2 }

SUB R6, R5, R4 { R6 $\leftarrow$ R5 - R4 }

Ans. 8 clock cycles

Q. The instruction pipeline of a RISC processor has the following stages: Instruction Fetch (IF), Instruction Decode (ID), Operand Fetch (OF), Perform Operation (PO) and Writeback (WB). The IF, ID, OF and WB stages take 1 clock cycle each for every instruction. Consider a sequence of 100 instructions. In the PO stage, 40 instructions take 3 clock cycles each, 35 instructions take 2 clock cycles each, and the remaining 25 instructions take 1 clock cycle each. Assume that there are no data hazards and no control hazards. How many clock cycles are required for completion of execution of the sequence of instruction?

Given, total number of instructions (n) = 100

Number of stages (k) = 5

Therefore, the number of clock cycles required = Total number of cycles required in general case + Extra cycles required (here, in PO stage),

$$\begin{aligned} &= (n + k - 1) + \text{Extra cycles} \\ &= (100 + 5 - 1) + 40 \cdot (3 - 1) + 35 \cdot (2 - 1) + 25 \cdot (1 - 1) \\ &= (100 + 4) + 40 \cdot 2 + 35 \cdot 1 + 25 \cdot 0 \\ &= 104 + 115 \\ &= 219 \text{ cycles} \end{aligned}$$

Q. The instruction pipeline of a RISC processor has the following stages: Instruction Fetch (IF), Instruction Decode (ID), Operand Fetch (OF), Perform Operation (PO) and Writeback (WB). The IF, ID, OF and WB stages take 1 clock cycle each for every instruction. Consider a sequence of 100 instructions. In the PO stage, 20 instructions take 3 clock cycles each, 30 instructions take 2 clock cycles each, and the remaining 50 instructions take 1 clock cycle each. Assume that there are no data hazards and no control hazards. Estimate how many clock cycles are required for completion of execution of the sequence of instruction?

Given, total number of instructions (n) = 100

Number of stages (k) = 5

Therefore, the number of clock cycles required = Total number of cycles required in general case + Extra cycles required (here, in PO stage),

$$\begin{aligned} &= (n + k - 1) + \text{Extra cycles} \quad (2 \text{ marks}) \\ &= (100 + 5 - 1) + 20 \cdot (3 - 1) + 30 \cdot (2 - 1) + 25 \cdot (1 - 1) \\ &= (100 + 4) + 20 \cdot 2 + 30 \cdot 1 + 25 \cdot 0 \\ &= 104 + 70 \\ &= 174 \text{ cycles (4 marks)} \end{aligned}$$

Consider a pipeline having 4 phases with duration 60, 50, 90 and 80 ns. Given latch delay is 10 ns. Calculate-

i) Pipeline cycle time, ii) Non-pipeline execution time, iii) Speed up ratio

iv) Pipeline time for 1000 tasks, v) Throughput

i) Pipeline cycle time T_p = Maximum delay due to any stage + Delay due to its register

$$= \text{Max} \{ 60, 50, 90, 80 \} + 10 \text{ ns}$$

$$= 90 \text{ ns} + 10 \text{ ns}$$

$$= 100 \text{ ns}$$

ii) Non-pipeline execution time for one instruction

$$= 60 \text{ ns} + 50 \text{ ns} + 90 \text{ ns} + 80 \text{ ns}$$

$$= 280 \text{ ns}$$

iii) Speed up ratio = Non-pipeline execution time / Pipeline execution time

$$= 280 \text{ ns} / \text{Cycle time}$$

$$= 280 \text{ ns} / 100 \text{ ns}$$

$$= 2.8$$

iv) Pipeline time (ET) for 1000 tasks = $(k+n-1)T_p$

$$= (4+1000-1) \times 100 \text{ ns}$$

$$= 100300 \text{ ns}$$

v) Throughput for pipelined execution = n/ET

= Number of instructions executed per unit time

$$= 1000 \text{ tasks} / 100300 \text{ ns}$$

Q.Consider an unpipelined instruction. 1 ns clock cycle; 4 cycles for ALU and branches; 5 cycles for memory operations; relative frequencies 40%, 20%, 40%; 0.2 ns pipeline overhead (e.g., due to stage imbalance, pipeline register setup, clock skew) Question: How much speedup by pipeline?

• **Answer**

$$\text{Avg instr time unpipelined} = \text{Cycle Time}_{\text{non-pipeline}} \times \text{avg CPI}$$

$$= 1 \text{ ns} \times [(0.4+0.2) \times 4 + 0.4 \times 5]$$

$$= 4.4 \text{ ns}$$

For 1 instn: Cycle Time_{non-pipeline} = Cycle Time_{pipeline}, = 1 ns

Pipeline overhead = 0.2 ns

$$\text{Avg instr time pipelined} = \text{Cycle Time}_{\text{pipeline}} + \text{overhead}$$

$$= 1 + 0.2$$

$$= 1.2 \text{ ns}$$

$$\text{Speedup by pipelining} = \text{Avg instr time unpipelined} / \text{Avg instr time pipelined}$$

$$= 4.4 \text{ ns} / 1.2 \text{ ns}$$

$$= 3.7 \text{ times}$$

Question



APJ ABDUL KALAM
TECHNOLOGICAL UNIVERSITY
modassurilal, ueomljam modinlal, uozowal

Consider the following sequence of instructions being processed on the **pipelined 5-stage RISC processor** discussed in class:

Load R4, #100(R2)

Add R5, R2, R3

Subtract R6, R4, R5

And R7, R2, R5

- (a) (6 points) Identify all the data dependencies in the above instruction sequence. For each dependency, indicate the two instructions and the register that causes the dependency.

Solution:

There are three data dependencies in this instruction sequence:

- (i) Subtract instruction depends on Load instruction for register R4
- (ii) Subtract instruction depends on Add instruction for register R5
- (iii) And instruction depends on Add instruction for register R5

Q. Consider the following sequence of instructions being processed on the pipelined 5-stage RISC processor.

Load R4, #100(R2)

Add R5, R2, R3

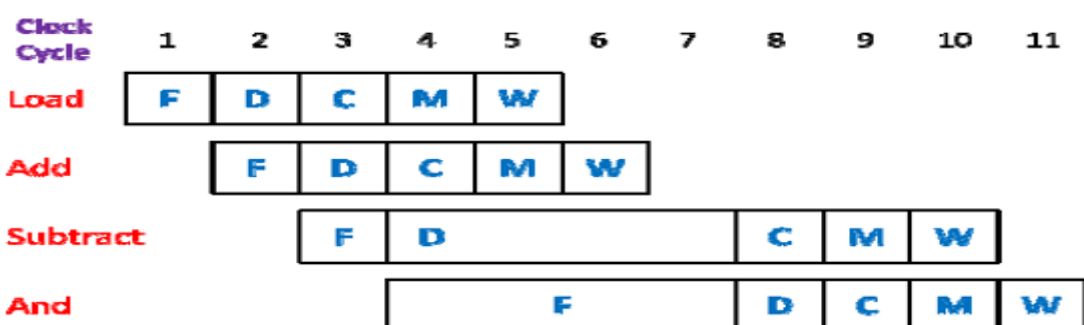
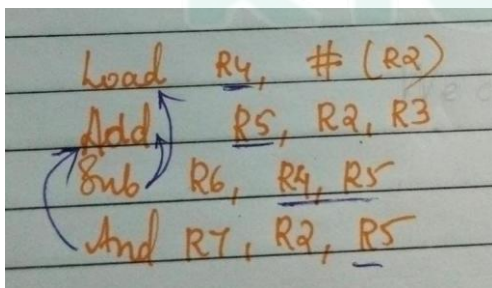
Subtract R6, R4, R5

And R7, R2, R5

Assume that the pipeline does not use operand forwarding. Also assume that the only sources of pipeline stalls are the data hazards. Draw a diagram that represents instruction flow through the pipeline with Fetch, Decode, Compute, Memory Access, Write Back phases during each clock cycle. The decode stage of Subtract instruction, takes 4 clock cycles. Analyse how long does it take for the instruction sequence to complete?

The following diagram shows the instruction flow through the pipeline. As shown in the diagram, the instruction sequence is completed in 11 cycles.

Dependencies of the above question:

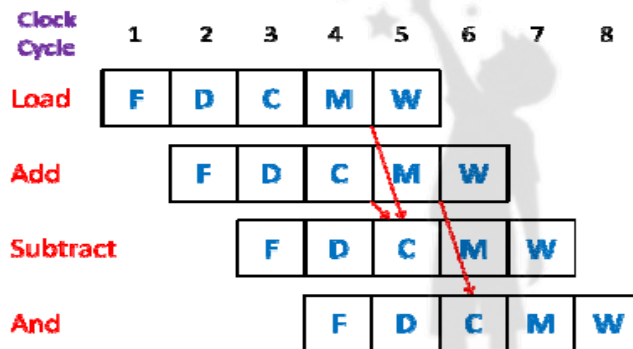


download from ktuspecial.in

- (c) (6 points) Now, assume that the pipeline uses operand forwarding. There are separate forwarding paths from the outputs of stage-3 and stage-4 to the input of stage-3. Draw a diagram that represents the flow of instructions through the pipeline during each clock cycle. Indicate operand forwarding by arrows.

Solution:

The following diagram shows the instruction flow through the pipeline in the presence of operand forwarding:



Consider an in-order MIPS multi-cycle pipeline that supports operand forwarding and executes the following instructions:

I1: LOAD F1, 0(R2)

I2: LOAD F2, 8(R2)

I3: FADD F3, F1, F2

I4: STORE 8(R2), F3

If the instruction fetch for I1 starts at clock cycle 1, in which cycle the instruction I4 access memory to store the data? (Here LOAD and STORE uses integer EX unit, and FADD uses a 4-stage adder. Instruction format used: opcode destination, source))

11th Cycle

	Cycle												
	1	2	3	4	5	6	7	8	9	10	11	12	13
I1: LD F1, 0(R2)	IF	ID	IE	MEM	WB								
I2: LD F2, 8(R2)		IF	ID	IE	MEM	WB							
I3: FADD F3, F1, F2			IF	ID	STALL	A1	A2	A3	A4	MEM	WB		
I4: SD F3, 8(R2)				IF	STALL	ID	STALL	STALL	STALL	EX	MEM	WB	

Dependencies of the above question:

Data Hazard: (Dependencies)

load	P1, 0(R2)
load	P2, 8(R2)
FADD	F3, P1, P2
store	8(R2), F3

(Note: In the original image, a blue arrow points from the 'load' instruction to the 'FADD' instruction, and another blue arrow points from the 'store' instruction to the 'FADD' instruction, indicating data dependencies.)



KTU SPECIAL CONNECT WITH US



WHATSAPP GROUP



FOLLOW US NOW



TELEGRAM CHANNEL



SUBSCRIBE NOW



www.ktuspecial.in

SUBSCRIBE



FOLLOW US



Visit Website

